The Architecture Axis Is Not the Axis: A Diagnostic Study of Neural Architecture Search and Score Fusion in Unsupervised Industrial Anomaly Detection

Hong Li[1], Yuanwu Zhou[1], Peng Deng[1*]

[1*]School of Civil and Architectural Engineering, Guangxi University of Science and Technology, No. 2 Wenchang Road, Liuzhou, 545006, Guangxi, China.

*Corresponding author(s). E-mail(s): dengpeng@gxust.edu.cn;
Contributing authors: lihong@gxust.edu.cn; zhouyuanwu@gxust.edu.cn;

Abstract

Two decisions set the accuracy of an anomaly detector: which network to deploy, and how the per-pixel map becomes the accept/reject score. The second is made silently — an unweighted mean over all pixels — and on the detector we study it is the one that matters. Changing only that reduction, on trained models, raises mean AUROC from 82.80% to 90.21% on MVTec-AD and from 82.35% to 90.57% on VisA, and lifts a category that otherwise fails below chance from 42.22% to 84.01%, with no retraining and no labels. The gain follows a dilution law: the mean scales with defect area, top-k with the tail floor, so it holds only while the defect covers under a tenth of the image and fades where the default saturates. The architecture axis contributes 0.54% of the variance in accuracy against 95.32% for the category, two thirds of a standard 42-cell search space cannot reach the deployed model, and twelve searches each returned a collapse-prone backbone. The benchmark cannot adjudicate either choice: against a 0.5-point margin, the standard comparison resolves 2.98. We report a pipeline at 82.47% where published methods reach 99%, and show where the missing points are, and are not.

Keywords: Industrial visual inspection, Unsupervised anomaly detection, Neural architecture search, Knowledge distillation, Score fusion, Proxy reliability

1 Introduction

Visual inspection on a production line has to work without defect samples. Collecting and annotating the failures of a manufacturing process is expensive, and the failure modes that matter are by definition the ones not yet seen, so the established approach is to model normality and flag deviations. Reverse distillation [1] is a representative and widely deployed member of this family: a frozen teacher extracts features at several scales, a student is trained to reproduce them on normal images, and at test time the discrepancy between the two is taken as the anomaly signal.

Two engineering decisions have to be made before such a system can be deployed. The first is the *architecture*: what network the student should be. The second is the *scoring policy*: how the per-pixel discrepancy maps are combined into the single number that decides whether a part is rejected. The second decision is usually made implicitly — the published pipeline averages the discrepancy map over all pixels, a choice that is

rarely stated and, to our knowledge, never justified. The literature has overwhelmingly invested in the first, searching over architectures on the argument that the optimal architecture is dataset-dependent and should be discovered rather than chosen [2, 3]; a pipeline of the kind we study inherits a search stage for exactly this reason.

The margins the field argues about, and the resolution of its benchmark. Progress on the standard benchmark is reported in tenths of a percentage point. PatchCore reached 99.1% image-level AUROC on MVTec-AD in 2022 [4]; the strongest methods reported since reach 99.6% [5, 6]. The contested margin is therefore 0.5 points, on a benchmark whose test sets hold 1725 images across all fifteen categories — between 42 and 167 per category, mean 115 [7]. Before asking which design decisions produce such margins, it is worth asking whether the standard evaluation design can resolve them at all. We measure that first, because the answer determines how every subsequent comparison in this literature should be read.

This paper is accordingly a *benchmark-and-evaluation study*, not a new detector. Work of this kind measures the evaluation practice of a field rather than adding a method, and it has clear precedents in this area. Yu et al. [8] compared architectures returned by several NAS algorithms against architectures drawn uniformly at random from the same search space, and found the search phase uninformative. IM-IAD [9] consolidated competing evaluation protocols for industrial anomaly detection onto one footing. Bouthillier et al. [10] showed that unacknowledged run-to-run variance invalidates comparisons in machine-learning benchmarks and derived how many runs such comparisons require; the same re-examination has since been carried out for streaming [11] and industrial time-series [12] anomaly detection. We continue that line into a deployed unsupervised image-anomaly-detection pipeline: we instrument it, we re-run its search, and we train the complete comparison it never ran.

What we instrument. Our own earlier pipeline, KG-NAS, is a three-stage system of this family: a search ranks 42 configurations (14 backbones $\times$ 3 feature dimensions) on normal data alone using a knowledge-distillation signal, and the winning configuration is then trained as a reverse-distillation detector. We ask a question this literature has largely assumed rather than measured: *how much accuracy does the architecture choice actually buy, is the search that selects it doing its job, and what can the benchmark that judges both actually resolve?* We answer by measurement: we instrument the search, we train all 14 backbones of its search space on all 15 MVTec-AD categories under one protocol — 210 detectors — and we sweep the scoring policy on frozen models.

The benchmark resolves 3 ***points; the field argues about*** 0.5***.*** Two-way decomposition of the 210-detector matrix leaves a residual standard deviation of 3.30 points, and the run-to-run standard deviation of a single detector at a fixed (backbone, category) is 1.96 points over three seeds. Comparing two backbones across the fifteen categories is therefore a paired test whose minimum detectable difference is 2.98 **points** — 75% of the entire 3.98-point spread of the fourteen backbones, and six times the margin that separates the 2022 state of the art from the strongest result reported since [4, 6]. To resolve 0.5 points on a single category would take at least 241 independent training runs per backbone. This is the paper's measurement about the benchmark design rather than about this pipeline.

Two thirds of the search space cannot reach the deployed model. The searched feature dimension enters the search through the student projector, so the search signal depends on it, but it does not reach the detector: in the deployment path the student encoder builds its bottleneck with a hard-coded width while the decoder is constructed with the requested one, and the final training entry point never passes the argument at all. The three values of the searched dimension therefore produce the identical deployed model for a given backbone. The 42-configuration space contains 14 distinct models; the other 28 cells are deployment-equivalent duplicates. The recorded searches behave as this predicts: across four independent runs the selected backbone is the same every time, while the selected dimension differs (Section 4.1).

The search signal does not distinguish architectures, and collapses for some of them. Calling the search's own evaluation method ten times per configuration gives two

results. Across the twelve well-behaved backbones the median score spans only 5.7% — the same order as the within-configuration noise of a single backbone — so the signal carries essentially no architecture-level information. And two backbones fail in a specific way: `efficientnet_b1` returns exactly 0.0 in 73% of evaluations and `mobilenetv3_small_100` in 13%, because the student diverges, every validation batch trips a hard threshold, and the score is then silently computed as an average over zero samples. Since the search minimises, 0.0 is the best score attainable anywhere in the space (Section 4.2); over the project's own record of 834 search trials the collapse occurs in 8.0% of them.

Architecture is detectable, negligible, and does not transfer. Training 210 detectors under an identical protocol and decomposing the resulting AUROC matrix attributes 0.54% of the variance to the backbone and 95.32% to the category, with the backbone effect statistically significant ($p = 0.039$) and practically irrelevant. The architecture the search selected ranks fifth of fourteen and is statistically indistinguishable from twelve of the thirteen alternatives. More consequentially, the ranking does not transfer: the mean correlation between two categories' orderings of the 14 backbones is $\rho = 0.215$, and in 31% of the 105 category pairs the categories disagree outright. Yet the protocol searches on a single category and applies the winner to all fifteen (Section 4.3).

The consequence is measurable, and the selection is systematically wrong. Running the search itself with independent seeds, the two backbones that collapse to zero are selected every time: in twelve runs the search returned a collapse-prone backbone in twelve, none in the top four of the space, at a probability of 7×10^{-11} under uniform sampling. Its mean AUROC (80.91%) sits below the mean of uniform random selection (81.65%), though that gap is smaller than the 2.98-point resolution this paper derives, so the defensible claim is the systematic selection failure rather than the gap. A design axis that cannot reach the model, ranked by a signal that does not discriminate, on a quantity that does not transfer, is selected — reliably — by a failure mode (Section 4.5).

The leverage is in the decision nobody documents. On frozen models, changing only the reduction that turns discrepancy maps into an image score moves mean AUROC from 82.80% to 90.21% — +7.42 points on the seed we report in full, and +8.07 and +7.78 on two further seeds — and choosing the policy per category reaches 93.72%. The pooling change needs no criterion and no label. We also test whether the policy can be chosen per category without any anomaly label, by perturbing held-out normal images with synthetic defects and selecting the policy that best separates perturbed from clean; on the same models that procedure reaches 88.59% and thus does *not* improve on the fixed top-k convention, whose own mean is 88.99% on that split. We report the decomposition rather than the larger number (Sections 5.2 and 5.5).

Contributions

1. **The resolvability of the standard benchmark design.** We anchor the noise floor of this task to the margin the field actually contests, and show that the benchmark as it is normally used resolves 2.98 points — the minimum detectable difference of the 15-category *paired* comparison that averaging over the available categories allows, against a contested margin of 0.5. A genuinely single-category, single-run comparison is worse still: it needs at least 241 independent training runs per backbone to resolve the same 0.5 points. Either way a backbone comparison reported from one run per configuration on this benchmark is not a measurement of the effect it claims. This is a measurement about the evaluation practice this benchmark induces, and it is our primary contribution.
2. **A diagnosis of the architecture axis.** Two thirds of a standard search space is deployment-inert; its search signal does not discriminate between architectures and degenerates to a score of exactly zero for a subset of them; the axis contributes 0.54% of the variance in detection accuracy against 95.32% for the category.
3. **A controlled comparison of a full search space.** 14 backbones × 15 categories under one protocol (210 detectors), which makes the negative result measurable rather than anecdotal.

4. **Evidence that a single-category search cannot work.** The backbone ranking does not transfer across categories ($\bar{\rho} = 0.215$; 31% of the 105 category pairs reverse), so a search run on one category estimates a quantity that does not exist independently of the category.
5. **The axis that does matter, and the law that governs it.** The scoring policy is worth 7.4 points of mean AUROC without retraining on MVTec-AD, 8.0 on a second student family and 8.2 on VisA (Section 5.3), and most of it is available from a fixed top-k pooling. The gain is not a free lunch but a consequence of dilution: the mean scales with the defect area, top-k with the tail floor of the map, so the two cross over at a defect size of a few percent, and the crossover predicts which categories lose. The label-free selection criterion we propose adds a mean of -0.40 points on top of that fixed convention, so we present the pooling as the finding and the selector as an unproven refinement.

Scope and honesty

We do not claim state of the art: the pipeline reaches 82.47% mean AUROC on MVTec-AD, against published values of 95.6%–99.6% for the established methods, and Section 6 reports that gap rather than obscuring it. The contribution is not a better number but a quantitative account of which design choices produce the number, of how much of the number the benchmark can measure at all, and of the finding that the axis the field searches is not the axis that matters. Section 2 positions the work, Section 3 describes the pipeline under study, Section 4 reports the diagnosis and the resolvability analysis, Section 5 the scoring-policy analysis, Section 6 states where the pipeline stands, and Section 7 concludes.

2 Related Work

2.1 Unsupervised Anomaly Detection

Unsupervised anomaly detection models normality and flags deviations, because defect samples are scarce and the failures that matter are the ones not yet seen. Four families dominate. Reconstruction methods learn to reproduce normal images and score by reconstruction error, from autoencoders scored with a structural similarity measure [13] to generative adversarial [14, 15] and memory-augmented formulations [16]. Feature-embedding methods characterise the distribution of pre-trained deep features, either parametrically as in PaDiM [17], by nearest-neighbour matching as in SPADE [18] and PatchCore [4], or by density estimation with normalizing flows [19]. Teacher–student methods distill a frozen teacher into a student that reproduces it on normal data, using multi-scale feature matching [20, 21] or, in reverse distillation, a decoder that reconstructs the teacher's features from a bottleneck embedding [1]; the present paper studies a pipeline of this last kind. Discriminatively trained methods inject synthetic defects [22] and have produced the strongest recent results [5, 6, 23, 24]; surveys of the area are available [25].

Two properties of this literature matter here. The architecture is treated as a given — either the backbone of a pre-trained feature extractor [4, 17] or a fixed encoder–decoder pair [1] — and the step that turns a per-pixel anomaly map into the single number on which a part is accepted or rejected is usually left implicit. The pipeline we study reduces the multi-scale discrepancy map by an unweighted mean over all pixels; Section 5 shows that on the detector we instrument this one undocumented choice is worth several times more accuracy than the architecture, and that it can be revisited without retraining.

2.2 Neural Architecture Search and the Evaluation of Its Search Phase

Neural architecture search automates the choice of network topology, from early evolutionary [3] and reinforcement-learning formulations [26, 27] through differentiable relaxation [28], platform-aware multi-objective search [29] and regularised evolution [30]; see [31] for a survey. Two lines of work are directly relevant.

The first applies NAS to anomaly detection. For image anomaly localisation, Qian et al. [2] generate anomalous samples with a knowledge-guided generator and use them as a reinforcement-learning reward, so that architectures can be compared on a validation set that would otherwise be unavailable. Work in this line takes for granted

that a better architecture will produce a better detector, and measures success by the downstream result alone.

The second line asks whether the search phase itself deserves that trust, and it is the immediate predecessor of this paper. Yu et al. [8] compared the architectures returned by several NAS algorithms against architectures sampled uniformly at random from the same search space, and found that, on average, the NAS algorithms perform *similarly to the random policy*; they attribute this to weight sharing degrading the ranking of candidates so that it no longer reflects true performance. Our findings agree in direction and go further in three respects. First, from "similar to random" to a *systematically wrong* selection: in Section 4.5 twelve independent runs of the search all return one of the two configurations prone to a degenerate evaluation, never one of the four strongest backbones, and the mean accuracy of the architectures they return sits at the 1.2th percentile of the distribution obtained by uniform sampling (80.91% against 81.65%, with the best backbone at 83.34%; the 0.74-point mean gap is itself below this paper's 2.98-point resolution, so we rest the claim on the selection behaviour rather than the gap). Second, a mechanism rather than a symptom: the cause is not weight sharing but an interaction between the objective and the selection rule, as the evaluation can silently return exactly 0.0 when every validation batch is rejected, and since the search minimises, that value is the optimum of the search's own range (Section 4.2). Third, structure rather than signal alone: two thirds of the search space cannot reach the deployed model at all (Section 4.1), and the quantity being ranked contributes 0.54% of the variance in accuracy against 95.32% for the category (Section 4.3). Where Yu et al. study image classification and language modelling with weight-sharing supernets, we study single-class unsupervised anomaly detection with an independent quick-training proxy, and we additionally show that the backbone ordering does not transfer between categories ($\bar{\rho} = 0.215$; 31% of category pairs reverse), which makes a single-category search unsound regardless of the size of the architecture effect.

2.3 Benchmark and Evaluation Studies

The present paper belongs to a class of work whose contribution is a measurement about how a field evaluates itself, rather than a new method. The class is established, and we adopt three of its methods.

Bouthillier et al. [10] showed that machine-learning benchmarks routinely compare algorithms on a single run, that the unacknowledged run-to-run variance is then large enough to reverse conclusions, and that the number of runs a comparison requires can be derived from the variance. Their analysis is the methodological template for Section 4.4: we estimate the residual variance of the task, fold in the externally reported effect size, and report the smallest difference the standard design can resolve. IM-IAD [9] performed the corresponding consolidation for industrial image anomaly detection, reproducing published methods under one protocol and showing how much of the reported spread survives standardisation. For anomaly detection outside the image domain the same re-examination has been carried out on streaming data [11] and, most recently, on industrial multivariate time series, where unified reproduction was combined with an event-level evaluation [12].

Our paper extends this line in two ways that we have not seen for unsupervised image anomaly detection. It supplies the fully crossed configuration $\times$ category comparison that a variance decomposition requires — 210 detectors, every backbone of a search space on every category of the benchmark, one protocol — which is what turns a diagnosis of the architecture axis into a measurement rather than an anecdote. And it anchors the resulting noise floor to the margin the field actually contests, so that the resolvability of the standard benchmark design can be stated in the same units as the claims it is used to support. Neither IM-IAD nor the time-series reproductions reports that quantity; for this task it turns out to be decisive.

2.4 Knowledge Distillation

Knowledge distillation transfers the behaviour of a strong model to a smaller one by matching outputs or intermediate representations [32], including similarity-preserving [33], structured [34] and self-distilled [35] variants and applications to detection [36, 37]. In anomaly detection, distillation is predominantly a *detection* device: the teacher–student discrepancy is the anomaly signal [1, 20, 21]. Using it as the *search metric* — ranking candidates by how well a student distils from a teacher, rather than by the accuracy of the detector it will eventually become — has been explored for NAS generally [38] and, for anomaly localisation, by Qian et al. [2].

That reuse is the premise our results call into question. Section 4.2 evaluates the distillation proxy directly, by repeating the search's own evaluation ten times per configuration, and finds that it separates the twelve degenerate-free backbones by only 5.7% of its median — the same order as its own run-to-run noise — while collapsing to an exact zero for two of them. Its ordering is uncorrelated with the true ordering of the same backbones: Spearman $\rho = -0.12$ over all fifteen categories and $\rho = 0.02$ on the very category the search is run on. The proxy is measuring something, but not the thing it is used to select for.

2.5 Positioning

This paper is a benchmark-and-evaluation study of a deployed design, not a new detector, and we claim no priority for combining knowledge distillation with architecture search. It differs from the NAS-for-anomaly-detection literature in asking whether the search phase works rather than assuming it; it extends the evaluation framework of Yu et al. [8] to unsupervised anomaly detection, quantifying the failure rather than merely detecting it; it adopts the variance discipline of Bouthillier et al. [10] and the protocol consolidation of IM-IAD [9], and reports the resolvability of the benchmark that both leave unstated; and it contributes a controlled comparison of a complete search space under one protocol (Section 6) which makes the diagnosis measurable. On the constructive side, it identifies the inference-time scoring policy — the reduction that turns a discrepancy map into an image score — as the axis that actually moves accuracy, worth 7.4 points on MVTec-AD and 8.2 on VisA without retraining, and it explains that gain with a dilution argument that also predicts on which categories the change loses (Section 5.3). It further tests a label-free procedure for choosing the pooling per category and reports that the procedure does not improve on the fixed pooling choice (Table 7).

3 Materials and Methods

This section describes the pipeline whose two design axes are examined in Sections 4 and 5. It is a three-stage knowledge-distillation pipeline built on reverse distillation [1]: a teacher is pre-trained on normal images, an architecture is searched using a distillation signal, and the winning architecture is trained as a reverse-distillation detector. Figure 1 sketches the three stages and marks the two axes this paper measures. We describe the pipeline precisely, because several of the findings that follow are properties of specific implementation decisions rather than of the method in the abstract. Stage 1 pre-trains a teacher encoder on normal training images with a contrastive objective. Stage 2 searches an architecture for the student, scoring each candidate by how well it distils from the frozen teacher on normal data using a small number of training epochs. Stage 3 trains the selected architecture — student encoder plus decoder — as a reverse-distillation detector. The search runs on a single category and its result is applied to all categories of the dataset.

3.1 Stage 1: Teacher Pre-training

Two teachers exist in this pipeline and they are not the same network. For the search signal of Stage 2, a ResNet-18 teacher with a 128-dimensional projection head is pre-trained for 100 epochs on the normal training images of the search category with a contrastive objective, and frozen thereafter; its role is to define the feature space that the proxy score compares against. The *deployed* detector of Stage 3, however, builds its teacher from ImageNet-1k weights [1, 39]: the `TeacherFeatureExtractor` instantiates a ResNet-18 initialised from `ResNet18_Weights.IMAGENET1K_V1`, freezes it, and never adapts it to the category being inspected. The two roles are worth separating

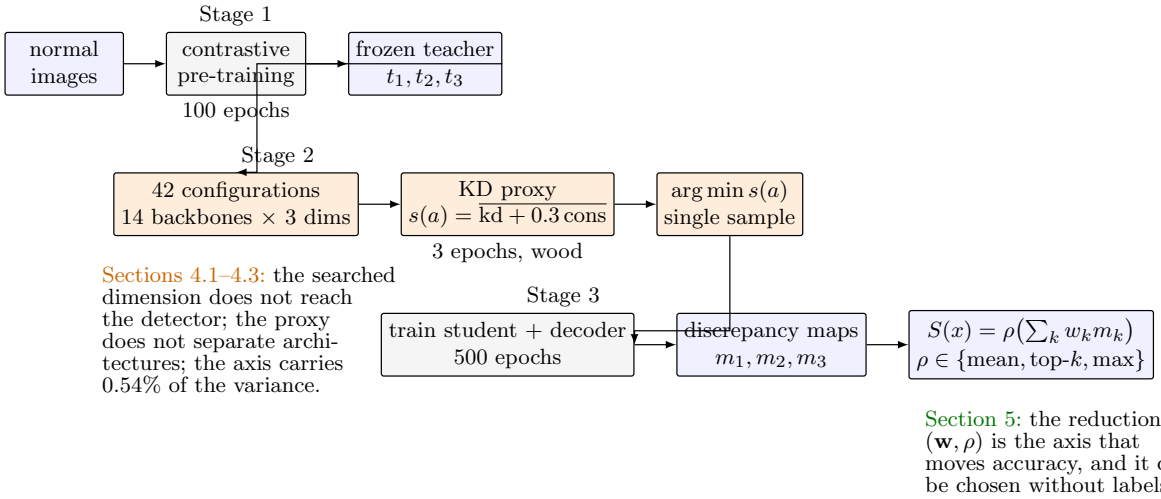


Fig. 1 The pipeline under study. A teacher is pre-trained on normal images (Stage 1); an architecture is selected from 42 configurations by minimising a short-training distillation proxy on one category (Stage 2); the selected architecture is then trained as a reverse-distillation detector (Stage 3) and each test image is scored by reducing its multi-scale discrepancy maps with a policy $(\mathbf{w}, \rho)$. This paper measures the two axes in bold: the architecture that Stage 2 selects, and the reduction that Stage 3 applies.

explicitly. Every detection result reported in this paper — the variance decomposition of Section 4, the policy comparison of Section 5, and the positioning of Section 6 — is produced by the *deployed* detector, and therefore rests on the ImageNet teacher; the contrastive teacher never scores an image for reporting and enters only through the Stage-2 proxy. The ImageNet teacher is a deviation from the reverse-distillation recipe, which pre-trains the teacher on the target category's normal images without labels; we report it here rather than leave it implicit, and Section 4.4 treats it as one candidate explanation for the gap to the published numbers of Section 6.

Whichever teacher is in use contributes intermediate features at three resolutions: the outputs of `layer1`, `layer2` and `layer3`, of shape $64 \times 56 \times 56$, $128 \times 28 \times 28$ and $256 \times 14 \times 14$ for an input resized to 224×224.

3.2 Stage 2: Architecture Search

The search space is the Cartesian product of 14 backbones and three feature dimensions, giving 42 configurations. The backbones span convolutional and attention-based families from 2015 to 2023, including ResNets [39], EfficientNet variants, MobileNetV3 and MobileOne, ViT and DeiT variants, Swin-Tiny, and the ConvNeXt and ConvNeXt V2 families [40]; Section 6 lists them with their deployed sizes. The three feature dimensions are 128, 256 and 512.

The evaluation signal

Each candidate is evaluated by building a student with the candidate's backbone and projection dimension, training it for a small number of epochs on the normal training images of the search category, and then measuring a composite distillation loss over validation batches $\mathcal{B}$ of held-out normal images:

$$\begin{aligned} s(a) &= \frac{\sum_{b\in\mathcal{B}} \mathbb{1}[\ell(b) < \tau]\ \ell(b)}{\max\left(\sum_{b\in\mathcal{B}} \mathbb{1}[\ell(b) < \tau]\,,\ 1\right)}, \\ \ell(b) &= \mathcal{L}_{\text{kd}}(b) + 0.3\,\mathcal{L}_{\text{cons}}(b), \end{aligned} \tag{1}$$

where $\mathcal{L}_{\text{kd}}$ is the mean-squared error between the student's projection and the teacher's, and $\mathcal{L}_{\text{cons}}$ is the mean-squared error between the student's projections of two independently augmented views of the same batch. A validation batch is accumulated only if its loss is finite and below a fixed threshold $\tau = 1000$; otherwise it is skipped and does not enter either the numerator or the denominator. Lower is better. Each trial constructs and trains a fresh student, and the protocol draws 100 trials

from a space of only 42 cells, so a cell is generally visited more than once and each visit re-trains a stochastically initialised student. Repeated visits therefore do give several independent scores for some cells; the search entry point, however, returns only the winning architecture, so the value it reports carries no variance estimate. The variance of the signal is estimated separately, from the repeated-evaluation study of Section 4.2. The two numerical consequences of the indicator in Equation (1) are examined in Section 4.2 and are stated here for completeness. If every batch of a trial is skipped the denominator is $\max(0, 1) = 1$ and the signal returns exactly zero, which is the minimum of the range of s. And because a skipped batch is excluded rather than counted, the returned value is the mean over the batches that survived, not over the batches that were drawn. Because the backbones produce features of different dimensionality while the teacher projects into a 128-dimensional space, a candidate whose projection dimension differs from 128 is additionally given a learned linear layer that maps the student's output into the teacher's space before the losses are computed; this alignment layer exists only during search.

The search draws candidates uniformly at random from the 42 cells and returns the one with the smallest score,

$$a^\star \;=\; \arg\min_{a \in \mathcal{A}_{\text{sampled}}} \; s(a), \tag{2}$$

and the number of draws is a protocol parameter (the recorded runs used 100, of which a variable number complete successfully).

3.3 Stage 3: Reverse-Distillation Detection

The selected configuration is re-instantiated as a detector. A student encoder takes the backbone's final feature map, adaptively pools it to 14×14, and projects it through a 1×1 convolution with batch normalisation and ReLU to a bottleneck of width B. A decoder then reconstructs the teacher's features in reverse order: a first block produces the 14×14 stage at 256 channels, the output is upsampled and passed through a second block producing the 28×28 stage at 128 channels, and upsampled again through a third block producing the 56×56 stage at 64 channels. We note for later use that the bottleneck width is fixed at $B = 256$ in the deployment path, independently of the feature dimension that the search may have selected; Section 4.1 shows that this makes the searched dimension unobservable in the deployed detector. The detector is trained on the normal training images of each category separately, with AdamW at a learning rate of 10^{-3} and weight decay 10^{-4}, a cosine schedule, and random horizontal flipping with probability 0.5; the loss is the sum of mean-squared errors between the student's reconstructions and the teacher's features at the three scales.

Anomaly scoring

Given a test image x, the teacher produces features t_1, t_2, t_3 and the student produces reconstructions s_1, s_2, s_3. For each scale k we form a discrepancy map

$$m_k(x) \;=\; \big\| s_k(x) - t_k(x) \big\|_2, \tag{3}$$

taking the L_2 norm across channels, and upsample each map to the input resolution. An image-level score is then obtained by combining the maps with non-negative weights $\mathbf{w} = (w_1, w_2, w_3)$ on the probability simplex and reducing the resulting map to a scalar with a pooling operator ρ:

$$\begin{aligned} S(x) \;&=\; \rho\big(\textstyle\sum_k w_k \, m_k(x)\big)\,, \\ &\rho \in \{\,\text{mean},\ \text{top-}k,\ \max\,\}. \end{aligned} \tag{4}$$

We refer to the pair $(\mathbf{w}, \rho)$ as the *scoring policy*. The published pipeline uses $\mathbf{w} = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ and $\rho = \text{mean}$, that is, an unweighted average over all pixels of the sum of the three maps. This choice is not forced by the method: given a trained detector the maps m_k are already computed, so any policy can be evaluated on the same model. Section 5 evaluates a fixed grid of policies drawn from Equation (4); we do not anticipate its result here, except to note that the policy is a design choice the search does not touch.

3.4 Protocol and Implementation Details

Unless stated otherwise, the detector above is trained for 500 epochs with batch size 64 and seed 42, with a ResNet-18 teacher, on the MVTec-AD

dataset [7]. The search uses three quick training epochs per candidate with batch size 16 on the `wood` category. Each search run performs 100 draws from the 42-cell space (Section 4.2); the recorded runs used 100, of which a variable number completed. Inputs are resized to 224×224 and normalised with ImageNet statistics. All figures report image-level AUROC averaged over the 15 categories. Where a backbone's memory footprint exceeds the device at batch size 64, training uses gradient accumulation with a smaller micro-batch, keeping the effective batch size at 64; the micro-batch used is recorded per run and this affects only the largest backbones.

3.5 Use of Generative AI in This Work

In accordance with the journal's disclosure requirement, we state the role of generative AI explicitly rather than only in the back matter. An AI assistant (a large language model) was used substantially. It wrote the code for the diagnostic experiments, the synthetic reproduction and the benchmark harness; it executed those experiments and produced the numerical results reported here; and it produced drafts of the manuscript text. It did not determine the research questions, the experimental design or the interpretation of the results. The authors specified the research questions, directed the work, verified every reported result against the recorded experimental outputs, and take full responsibility for the content, the claims and their accuracy. No figure or table in this paper was generated by a generative model. Every figure is a direct plot of recorded measurements, produced by the released script `make_figures_v2.py` from the released JSON result files, and every table is a transcription of the same files; the AI assistant wrote the plotting code and ran it, but drew no image content and invented no data. All code, logs and derived artefacts are released so that every number can be traced to the run that produced it (Section 7).

4 Results: What the Architecture Search Actually Optimises

The three-stage pipeline of Section 3 treats the Stage-2 search as the component that produces the architecture, and the reported accuracy as evidence that the search worked. This section tests that reading directly. We ask four questions in turn: does the searched quantity reach the deployed model (Section 4.1), is the search signal stable enough to be ranked (Section 4.2), does the axis being searched have any measurable effect on detection accuracy at all (Section 4.3), what does the search actually return when it is run (Section 4.5), and — the question that bounds all the others — how large a difference can the benchmark that judges this pipeline resolve in the first place (Section 4.4)?

4.1 Two Thirds of the Search Space Cannot Reach the Deployed Model

The search space is $\mathcal{A} = \mathcal{B} \times \mathcal{D}$ with $|\mathcal{B}| = 14$ backbones and $\mathcal{D} = \{128, 256, 512\}$ feature dimensions, giving 42 configurations. The dimension enters the search through the student projector, `ContrastiveModel(projection_dim=feature_dim)`, so the proxy score does depend on it. It does not, however, reach the detector. The deployed detector is assembled in two stages, and both ignore the searched value. In the student encoder the bottleneck width is hard-coded: `StudentEncoder.forward` calls `_init_bottleneck(f.shape[1], 256, ...)`, so the encoder always produces a 256-dimensional bottleneck. The decoder is declared with a parameter, `Decoder(bottleneck_dim=...)`, but the final training entry point constructs the model as `RDDetector(student_backbone=..., teacher_arch=...)`, never passing `bottleneck_dim`; the decoder therefore keeps its default of 256 as well. Every stage that actually runs is pinned to 256.

Two consequences follow. First, the axis is unreachable rather than merely unused: passing the searched value through would not activate it, because the encoder half does not accept the

Table 1 The four recorded searches: trials declared and recorded, and the configuration the search returned. The backbone is the same in all four runs while the feature dimension varies, which is what an unreachable dimension predicts (Section 4.1).

Run	Trials (declared / recorded)	Selected backbone	Selected dim
stage2	30 / 17	efficientnet_b0	128
stage2_rtx3090	30 / 15	efficientnet_b0	512
stage2_rtx3090_100trials	100 / 39	efficientnet_b0	256
stage2_topjournal_15backbones_fixed	100 / 93	efficientnet_b0	256

argument at all — the two stages are wired independently and only the encoder's hard-coded 256 is on the training path. Second, all three values of the searched dimension produce the *identical* deployed detector for a given backbone: the 42-configuration space contains only 14 distinct models, so the remaining 28 configurations are deployment-equivalent duplicates of them.

The recorded runs behave as this analysis predicts. Across four independent searches on the same data and the same space, the selected backbone is `efficientnet_b0` every time, whereas the selected feature dimension differs between runs (Table 1): The stable part of the outcome is the part that matters for deployment; the unstable part is precisely the part that cannot affect it. The search expends two thirds of its ranking capacity on a variable with no deployment meaning, and reports the winner of that sub-ranking as a discovered design choice.

A second observation from Table 1: the number of trials recorded is systematically below the number declared. The search loop catches exceptions and `continue`s without appending a result, so failed trials vanish from the record and from the denominator of any success statistic. (The run directory named `15backbones` is a frozen label from an earlier draft of the experiment; the space it actually searched, and the space analysed throughout this paper, contains fourteen backbones.)

4.2 The Proxy Carries Almost No Architecture Signal, and Collapses to Zero

The proxy is the average over validation batches of kd + 0.3 consistency, and the candidate with the *lowest* score is selected. Two properties of the scoring code matter. A validation batch is accumulated only when `isfinite(score)` *and* `score < 1000`; a rejected batch is skipped with a warning. And the average is formed as `total_score / max(count, 1)`: if every batch in a trial is rejected then `count` is zero, and the function returns exactly `0.0` — which, because the search minimises, is the best score attainable anywhere in the space. A trial that failed to produce a usable number is thus indistinguishable from a trial that produced the best possible number.

Table 2 Proxy repeatability, condensed. The search's own evaluation method is called ten times for each of the 42 configurations on MVTec wood (420 evaluations). "Zero" is the fraction of repeats returning exactly 0.0; "CV" is the within-configuration coefficient of variation, pooled over each backbone's three dimensions. The two collapse-prone backbones are shown individually; of the remaining twelve not one degenerate trial occurred in any repeat. All twelve degenerate-free backbones have pooled medians spanning 7.538–7.971 with CV between 2.2% and 12.1%, except `efficientnet_b0`, whose pooled CV is 74% because its repeats are degenerate-free but wide. Their per-backbone values are released with the code.

Backbone	Repeats	Zero (%)	Median proxy	CV (%)
efficientnet_b1	30	**73**	**0.000**	169.5
mobilenetv3_small_100	30	**13**	7.979	180.9
twelve degenerate-free backbones (each)	30	0	7.538–7.971	2.2–74.0
all 42 configurations	**420**	**6.2**	—	—

To measure how often this happens, and what the proxy does the rest of the time, we call the search's own evaluation method (`evaluate_architecture`) on MVTec wood — the category the search uses — and repeat every one of the 42 configurations ten times with independent seeds. Table 2 gives the result, and Figure 2 the same data.

Two regimes appear, and both matter. Twelve of the fourteen backbones are well behaved in the sense that matters for the search: not a single degenerate trial occurred in any repeat, so arg min never returns the spurious optimum. Within those twelve, all but `efficientnet_b0` have within-configuration coefficients of variation between 2.2% and 12.1%; `efficientnet_b0` is degenerate-free but noisy, with a pooled CV of 74%, so it sits at the upper edge of the CV range while its pooled median (7.971) sits at the top of the median range. We therefore report the median span over the twelve as 7.538–7.971, and note that excluding `efficientnet_b0` leaves the low end unchanged at 7.538 (ViT-Tiny). Two backbones are broken, and severely so: `efficientnet_b1` returns exactly 0.0 in 73% of evaluations at every dimension, and `mobilenetv3_small_100` in 13%. Across all 420 evaluations, 26 (6.2%) returned exactly zero. The cause is visible in the engine's

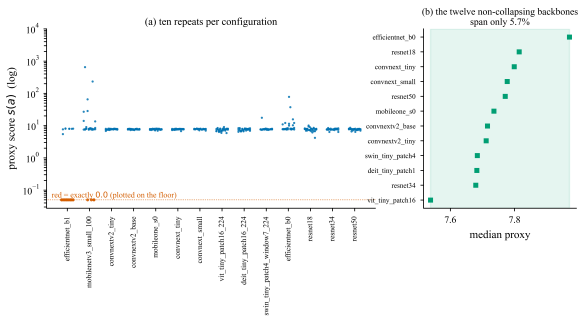


Fig. 2 Proxy repeatability. (a) Ten independent evaluations of each of the 42 configurations, on a logarithmic axis. Twelve backbones form a tight band near 7.6 (`efficientnet_b0` is degenerate-free but sits at the top of the band with the widest spread); `efficientnet_b1` and `mobilenetv3_small_100` intermittently return exactly 0.0 (marked red, plotted on the axis floor) when their student diverges. (b) The pooled median of the twelve degenerate-free backbones spans only 5.7%, the same order as the within-configuration noise of a single backbone.

own diagnostics — the student diverges, and every validation batch then trips the hard threshold:

```
Warning: Invalid score detected (score=4.19e+28, kd=2.18e+28), skipping
```

The proxy does not distinguish architectures.

The more consequential number in Table 2 is the median column. Across the twelve degenerate-free backbones — CNN and Transformer families, 2015-era through 2023-era, from ResNets [39] to ConvNeXt V2 [40] — the median proxy spans only 7.538 to 7.971, a range of 5.7%, while a single backbone's within-configuration noise is of the same order for most of them (Table 2 gives the per-backbone coefficients of variation; all but `efficientnet_b0` are below 13%, and `efficientnet_b0` itself is 74%). The proxy contains essentially no architecture-level signal. Any ranking it produces is not a ranking of architectures; it is a ranking of which cell happened to be sampled.

Minimising then selects the broken cells. This turns a noisy score into a systematically wrong one. A well-behaved configuration has a lower bound near 7.5 and can never beat a configuration that occasionally returns 0.0. Minimising over the space does not select the best candidate; it selects the candidate with the heaviest left tail — and the left tail is heaviest exactly where training diverged. Consistent with this, `efficientnet_b1`, the backbone that collapses in 73% of evaluations, is the arg min reported by the capacity searches of Section 4.3 at two of their five parameter bands.

Table 3 Degenerate trials in the recorded searches, and whether the plain arg min lands on one. A trial scoring exactly 0.0 has had every validation batch rejected, so the average is `total/max(count,1)`; because the search minimises, such a trial wins outright.

Run	Trials	Zeros	arg min config
`cap_budget_3M`	20	1	`mobilenetv3_small_100@128`
`cap_budget_6M`	40	4	`mobilenetv3_small_100@256`
`cap_budget_8M`	80	11	`efficientnet_b1@256`
`cap_budget_13M`	80	13	`mobilenetv3_small_100@256`
`cap_budget_30M`	80	6	`efficientnet_b1@256`
`mvtec_stage2_test/bottle`	100	9	`efficientnet_b1@512`
`search_strategies` (9 runs)	270	23	8 of 9 degenerate
`stage2` (main search)	17	0	`efficientnet_b0@128`
`stage2_rtx3090`	15	0	`efficientnet_b0@512`
`stage2_rtx3090_100trials`	39	0	`efficientnet_b0@256`
`stage2_topjournal_15backbones_fixed`	93	0	`efficientnet_b0@256`
Total	**834**	**67 (8.0%)**	**14 of 19 runs**

The recorded runs show the same pathology at the same rate. Over every search result stored in the project — 19 runs, 834 trials — 8.0% of trials score exactly zero, and in **14 of the 19 runs (74%)** the plain arg min over the recorded results, which is literally what the search returns (`min(self.results, key=score)`), points at a zero-score trial (Table 3). The five capacity-budget searches are all affected, and were usable only because a later analysis script discards trials scoring ≤ 0: three different conventions for handling degenerate trials coexist in the same codebase, the search engine keeping them while the downstream selectors drop them.

A reproducibility caveat. The recorded runs predate the current revision of the search engine, and the two are not numerically identical: the recorded proxy for `efficientnet_b0` spans 0.002–0.065, whereas the current code returns 7.5–12.2 for the same configuration. We therefore do not claim to reproduce the recorded search; what the two lines of evidence agree on — and all we rely on here — is the failure mode, its rate (8.0% against 6.2%) and the backbones it afflicts. The revised engine also returns no zero-score trial in the four runs whose results this paper reports, which is itself evidence of the revision; that the four happened to select `efficientnet_b0`, a backbone that does not collapse, was in this account luck rather than method (Section 4.5).

Two further obstacles. The proxy's absolute level is not comparable across categories: its median is 0.110 on `capsule`, 0.385 on `hazelnut` and 7.56 on `wood`, a span of 68.6×, so neither a

Table 4 Parameter-capacity bands. Each band restricts the search to backbones whose deployed size falls inside the band; the selected backbone is then trained with the identical protocol on all 15 MVTec-AD categories.

Band	Selected backbone	Params (M)	Mean AUROC (%)
0–3M	MobileNetV3-Small	2.81	82.01
3–6M	EfficientNet-B0	5.41	83.14
6–9M	DeiT-Tiny	7.31	81.55
9–15M	ResNet-18	13.04	**83.25**
15–31M	ResNet-50	25.77	83.08

fixed threshold nor a cross-category comparison of scores is meaningful — yet the protocol runs a single search on one category and transfers the winner to the others. And the deployment-inert dimension of Section 4.1, which cannot change the detector at all, nevertheless moves the proxy by a median factor of 1.16× across 140 backbone comparisons and by more than 10× at the 90th percentile. A variable with no deployment meaning is responsible for order-of-magnitude swings in the ranking.

4.3 The Searched Axis Has No Measurable Effect on Accuracy

A search that ranks noise could still be worth running if the axis it explores mattered. We test that directly, by retraining detectors across the whole parameter range under five mutually exclusive parameter bands, each trained with the identical protocol (500 epochs, batch size 64, seed 42, ResNet-18 teacher, reverse distillation). Each band is drawn from the candidates that a budget-constrained search had already evaluated, and within the band only backbones whose deployed size (encoder plus decoder) falls inside it are ranked; the best-scoring such backbone is then trained on all 15 MVTec-AD categories. Table 4 reports the result.

Multiplying the parameter budget by 9.2× changes mean AUROC by +1.07 percentage points, and the entire spread across the five bands is 1.70 points. The ordering is not monotone: the largest model is worse than the second largest. All ten pairwise paired t-tests across categories are non-significant (smallest $p = 0.058$; uncorrected for multiplicity across the ten contrasts, and no contrast would survive a Bonferroni or Benjamini–Hochberg correction, so the family-level conclusion is merely reinforced).

We then measured the axis directly: all 14 backbones of the search space were trained on all 15 MVTec-AD categories under the identical protocol, giving 210 detectors at seed 42, and the resulting matrix was decomposed. A two-way decomposition attributes 0.54% of the variance to the backbone ($F(13, 182) = 1.844$, $p = 0.039$; a two-way fixed-effects ANOVA on the *single-seed* matrix, with the backbone × category interaction as the error term, testing whether the fourteen backbone means are equal), 95.32% to the category, and 4.13% to their interaction, which is the error term in this repeated-measures design (Figure 3).

Two points about how these shares are formed. They are the ANOVA sum-of-squares shares of one matrix — the seed-42 run of each cell — and not averages over the three seeds; seeds enter the analysis only through the variance estimates of Section 4.4, where they appear as the pooled within-cell standard deviation. Reporting the shares as point estimates (0.54% and 95.32%) is therefore a statement about one fully crossed matrix, and the qualitative conclusion (architecture ≪ category) is what we carry forward; we have not put a confidence interval on the shares themselves, because doing so would require the replicated design to be fully crossed in seeds, which it is not (only six of the fourteen backbones were replicated at all three seeds; Section 4.4).

The backbone term is statistically significant: with 210 detectors the test has the power to detect it. That, precisely, is the point. The effect is detectable *and* negligible. Architecture accounts for 0.54% of the variance in detection accuracy, against 95.32% for which category is being inspected — a factor of 175. A practitioner choosing between two backbones is moving an AUROC whose variation is overwhelmingly determined by the object under inspection rather than by the network.

The ranking says the same thing. Over the 15 categories the 14 backbones span

3.98 points of mean AUROC, and the eleven strongest span only 2.43 points. The selected architecture, `efficientnet_b0`, ranks fifth of fourteen and is statistically indistinguishable from twelve of the thirteen alternatives; only `convnextv2_base` is significantly worse (−3.11 points, $p = 0.013$). That p is the smallest of the thirteen backbone-versus-backbone contrasts and is not corrected for multiplicity; at the 13-contrast family level it would not fall below a 0.05 threshold

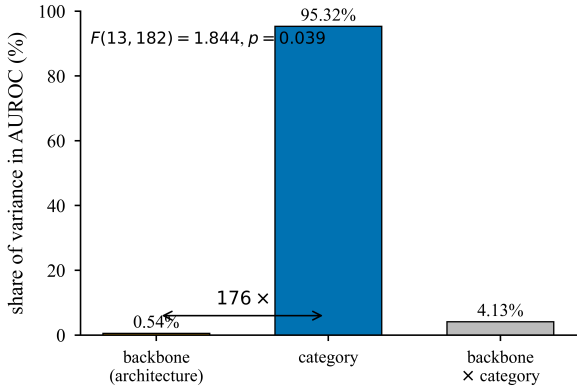


Fig. 3 Two-way decomposition of the true AUROC over 14 backbones and 15 categories (210 detectors). The backbone effect is statistically detectable and practically negligible: it accounts for 0.54% of the variance against 95.32% for the category, a factor of 175.

under a Bonferroni correction ($0.05/13 = 0.0038$). We therefore report it as the single contrast that reaches nominal significance, and we do not rest any claim on it — the claim we do make, that the backbone cannot be *ranked* by a comparison of this size, is supported by the resolvability calculation of Section 4.4 independently of any individual p.

The search signal does not predict the target. Ranking the 14 backbones by their mean proxy score (Section 4.2) and comparing that ordering with the true performance matrix gives a Spearman correlation of -0.12 ($p = 0.68$) against the 15-category mean, and $+0.02$ ($p = 0.93$) against performance on `wood` itself. The search signal is not weakly correlated with the outcome it exists to select for; on the very category the search is run on it is indistinguishable from uncorrelated. The consequence is concrete: on `wood` the backbone with the highest true AUROC is `convnextv2_base` at 94.74%, and across the fifteen categories `convnextv2_base` is the *worst* of the fourteen at 79.36%; the correlation between a backbone's rank on `wood` and its rank overall is -0.24. The single category the protocol searches on is, if anything, mildly anti-informative about the rest.

The run-to-run noise is not small. Within a fixed (backbone, category) cell the pooled standard deviation across training seeds is 1.96 points over 180 residual degrees of freedom, against a total spread of 3.98 points across the fourteen backbones. That the two numbers are of the same order is the whole difficulty of this axis: a single training run cannot resolve the backbone effect for any individual category, so a comparison built on one run per configuration is unresolvable in principle rather than merely imprecise. Section 4.4 turns that into a number.

And the ranking does not transfer. Correlating the 14-backbone orderings of every pair of categories (105 pairs) gives a mean Spearman $\rho = 0.215$, and in 31% of pairs the two categories *disagree* on the ordering ($\rho < 0$). The correlation is not an artefact of including the two collapse-prone backbones: excluding them raises it to $\rho = 0.256$ ($t = 6.0$ over the pairs), and excluding `efficientnet_b0` as well raises it further to 0.278 ($t = 6.6$). The transfer is therefore positive and statistically clear, and still too small to make a single-category search sound, because a third of the category pairs reverse the ordering outright. The protocol searches on a single category, `wood`, and applies the winner to all fifteen. Since which backbone is better depends on the category, a single-category search is not a noisy estimate of a real quantity; it is an estimate of a quantity that does not exist independently of the category. The three estimates, with their t statistics, are emitted by `proxy_diag/rho_wellbehaved_control.py`.

This is the central observation of the paper. Taken with Sections 4.1 and 4.2, it says that the proxy varies enormously across the space while the quantity it is supposed to predict contributes half a percent of the variance that the category contributes ninety-five percent of: the search ranks one noisy quantity in order to predict another that is almost entirely determined by something else.

4.4 What This Benchmark Design Can Resolve

The preceding subsections diagnose the search. This one asks a different question, about the benchmark rather than about the pipeline: *how small a difference can the standard evaluation design detect?* The question is forced by the state of the field. On MVTec-AD, PatchCore reached 99.1% image-level AUROC [4] and the strongest methods reported since reach 99.6% [5, 6]; the contested margin between successive generations is 0.5 points, on test sets holding 42–167 images per

category (mean 115) [7]. If published comparisons cannot resolve differences of that size, the ranking they produce is not a ranking of methods.

The 210-detector matrix of Section 4.3 and the replicated cells within it supply every quantity the calculation needs. Three numbers come out of them, all in AUROC points and all computed by `proxy_diag/samplesize_analysis.py`:

- $\sigma_{\text{seed}} = 1.96$, the pooled within-cell standard deviation over training seeds (90 cells replicated three times; 180 residual degrees of freedom). The distribution of per-cell standard deviations is heavily skewed — median 0.58, maximum 7.74 — so the pooled figure is the honest single number and the median understates the tail.
- $\sigma_{\text{res}} = 3.30$, the two-way ANOVA residual on the single-seed matrix. With one observation per cell this residual *is* the backbone × category interaction, which is why it is the working error for a comparison *between* backbones rather than the seed-only figure.
- $\sigma_{\Delta} = 4.12$, the empirical standard deviation of the per-category difference between two backbones, averaged over the 91 backbone pairs. This is what a paired comparison across categories actually sees.

The three quantities are related, and stating the relation removes an apparent inconsistency. If two backbones' deviations within a category were independent, a paired difference would have standard deviation $\sqrt{2}\,\sigma_{\text{res}} = 4.66$. The measured $\sigma_{\Delta} = 4.12$ is smaller, which corresponds to a within-category residual correlation of $\rho_{\text{bs}} = 0.221$ between two backbones: a category that is hard for one backbone tends to be hard for the others, so the two deviations move together and their difference is less variable than independence would imply. This is a property of the data rather than an error, and it is why the paired 15-category design is the more resolvable of the two we consider. All three numbers are emitted by `proxy_diag/samplesize_analysis.py`.

Two comparisons follow, and both are unfavourable to the design in common use. On a *single category*, with two backbones trained independently, the difference has resolution $\sigma_{\text{seed}} = 1.96$ per arm; detecting the 0.5-point contested margin at 80% power with a two-sided test at $\alpha = 0.05$ requires **at least** 241 **independent training runs per backbone** (240.2 before rounding up). Paired instead across the fifteen categories the benchmark provides, the minimum detectable difference is 2.98 **points** — 75% of the entire 3.98-point spread of the fourteen backbones, and six times the margin the field argues about. Adding two more training seeds per cell, as we do here, moves this only to 2.95 points. To resolve 0.5 points by pairing over categories would take at least 533 categories (532.2 before rounding up); the benchmark has fifteen.

The requirement is also method-specific, which is a further reason it has to be reported rather than assumed. Running the same calculation on a PatchCore-style memory-bank detector, benchmarked through the harness released with this paper, gives a seed standard deviation of 0.51 points and a requirement of only 17 runs per backbone — a factor of 14 below the reverse-distillation pipeline studied here. No single number of runs is therefore sufficient in this field; the figure belongs with the method it describes.

The implication is not that this benchmark is worthless, but that a single-category, single-run comparison on it cannot support a claim at the resolution the literature reports. A backbone ranking must either be averaged over many categories — which is what Table 8 does, and why that table needed 210 detectors — or be reported with an explicit statement of what it can and cannot distinguish. This is also why the architecture effect, though statistically detectable at 210 detectors ($p = 0.039$), cannot be *ranked* reliably by any smaller design: the power that makes the effect visible is the same power the field's usual single-category design lacks. We adopt this discipline for the rest of the paper, making every backbone comparison on the 15-category mean and reporting differences smaller than about three points as unresolved rather than as rankings.

4.5 Running the Search Selects the Broken Backbones

The preceding subsections describe a search space that is two thirds inert, a signal that does not discriminate between architectures and collapses to a score of exactly zero for two of them, and a target quantity that architecture barely moves. This subsection asks what they

amount to in practice, by simply running the search. We invoke the search engine's own entry point (`NASSearchEngine.search`) with the settings used throughout this paper — 100 trials, three quick training epochs per candidate, uniform sampling from the 42 cells — and repeat it with twelve independent seeds. Because the searched feature dimension cannot reach the detector (Section 4.1), the deployed model depends only on the selected backbone, so the true quality of a run's output can be read directly off the truth matrix of Section 4.3. Every run returned one of the two backbones that collapse to zero: eight of the twelve returned `efficientnet_b1` (true rank 12 of 14, 80.43% mean AUROC) and four returned `mobilenetv3_small_100` (rank 7 of 14, 81.85%). Of the fourteen backbones in the space, only these two were ever *returned* as the winner by any of the twelve runs; the other twelve were sampled but never selected. Under uniform selection the probability of twelve runs all landing inside that two-element subset is 7.2×10^{-11}. The search does not merely fail to find a good architecture; it is reliably drawn to the two configurations that break.

Two clarifications are needed here, because an earlier draft of this work reported a different set of runs and the two lines of evidence can look contradictory if their populations are not stated. First, *what is sampled versus what is selected*: every run evaluates a large fraction of the space — across the twelve runs the sampler drew all 14 backbones and between 34 and 42 of the 42 configurations — so the concentration in the selected set is a property of the selection rule, not of a restricted sampler. Second, *which run population*: the four searches discussed in Section 4.1 (the ones whose results this paper reports, and which predate the current revision of the engine) selected `efficientnet_b0`, a backbone of true rank 5 that collapses only rarely. The twelve runs here are the later, revised-engine repeats. The two groups therefore describe the same failure mode at different rates rather than contradicting each other: the four older runs happened not to draw a degenerate evaluation for their winner, and Section 4.2 documents that the revised engine returns no zero-score trial in those four runs at all. We report both populations explicitly for that reason, and the claim that follows rests on the twelve-run record, which is the larger and the more recent.

The consequence is quantitative, and it is not marginal (Figure 4). Averaged over the twelve runs the search's output attains **80.91**% mean AUROC. Picking a backbone uniformly at random from the same space gives **81.65**% (bootstrap s.d. 0.32 points), and the best backbone in the space gives **83.34**%. The search result sits at the **1.2th percentile** of the random-selection distribution — it is worse than 98.8% of uniform draws — with a mean true rank of 10.3 of 14 against 7.5 for random, and it never once selected one of the four strongest backbones. Two caveats on the size of this effect, both of which we prefer to state. First, the 0.74-point mean difference between the search's output (80.91%) and the random mean (81.65%) is smaller than the 2.98-point minimum detectable difference this paper derives for a paired comparison (Section 4.4) — so the defensible statement is not "the search is worse than random by 0.74 points" but "the search does not beat random, and its selection behaviour is systematically wrong": it returned a collapse-prone backbone in twelve of twelve runs — eight times `efficientnet_b1` (true rank 12 of 14) and four times `mobilenetv3_small_100` (rank 7) — so every one of the twelve is of true rank 7 or worse, and none is in the top four, at a probability of 7.2×10^{-11} under uniform sampling. Second, the percentile is from a bootstrap over 200 000 uniform draws of a backbone, whereas the 2.98 figure is a paired minimum detectable difference over the fifteen categories; the two are different statistics on different nulls and we do not compare them. It is that systematic error, not the 0.74-point gap, that the evidence supports, and it is sufficient: replacing the search with a random number generator would have improved the deployed detector; replacing it with the constant `resnet18` would have improved it by 2.3 points.

The mechanism is the one established in Section 4.2: a well-behaved configuration is bounded below by roughly 7–8, while the two collapse-prone backbones return exactly 0.0 whenever their student diverges, so minimising a single sample per configuration selects the heaviest left tail in the space — and the heaviest left tails belong to the configurations that failed. More trials make this more certain rather than less, because every additional sample of a collapse-prone backbone is another chance to return the minimum of the space.

The control that isolates the mechanism is to re-run the selection on the *same* twelve recorded trial sets with trials scoring exactly 0.0 filtered out, changing nothing else. The twelve runs then select `deit_tiny` (eight runs, true rank 6 of 14), `resnet18` (two, rank 1) and `efficientnet_b1` (two, rank 12) — a spread that includes the best backbone in the space and one near its bottom, but in no case the degenerate-value artefact that the unfiltered arg min returned. The filter does not guarantee a good winner; it guarantees that the winner is a real evaluation rather than a 0.0 produced by divergence. The failure is therefore the interaction of the degenerate value with the minimising rule, and not an intrinsically bad search space: the space contains reasonable winners and the sampler reaches them, but the plain arg min over unfiltered scores does not select them. This is why we phrase the recommendation as a filter rather than a blacklist, and it is also why the recommendation is not specific to the two backbones we observed collapsing. The control is released as `search_repeats/degenerate_filter_control.py`.

The exclusion of the two collapse-prone backbones is not perfectly clean: in the twelve recorded runs `efficientnet_b0` also returned exactly 0.0 twice, in 2.5% of its 79 trials, against 82% of 72 for `efficientnet_b1` and 22% of 87 for `mobilenetv3_small_100` in the same runs. The two extremes dominate the minimum, but no backbone is immune, which is consistent with the filter being the right instrument. Our recommendation therefore follows from the mechanism rather than from any single run: **a search whose objective can silently return the optimum of its own range must filter degenerate evaluations before minimising, and should report the distribution of its per-configuration scores rather than a single minimum.**

Why the earlier searches looked successful.

The four search runs whose results this paper reports selected `efficientnet_b0` (Section 4.1), a backbone that collapses only rarely and never in those four runs. That this is luck rather than method is the point of Section 4.5, and it is the reason we report both lines of evidence rather than either alone.

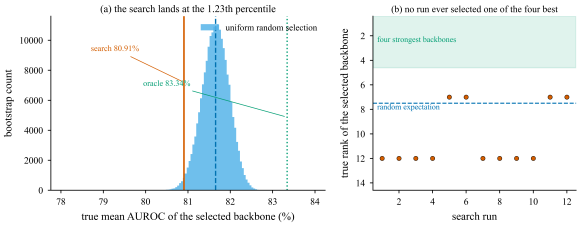


Fig. 4 The search against chance. (a) Bootstrap distribution of the true mean AUROC obtained by selecting a backbone uniformly at random from the same space, with the search's output and the oracle marked. The search result sits at the 1.2th percentile. (b) True rank of the backbone returned by each of the twelve runs: the four strongest backbones were never selected.

5 Results: Where the Leverage Is — the Inference-Time Scoring Policy

Section 4 established that the design axis the pipeline searches—the backbone—does not measurably move accuracy, while the search signal used to choose within it is unstable. That raises an obvious question: which design choices *do* move accuracy? This section shows that the way per-pixel discrepancies are combined into an image-level score—a choice that requires no additional training to revisit—is worth an order of magnitude more than the architecture.

5.1 The Scoring Policy as a Design Axis

With a trained detector the anomaly score of an image is formed in two steps (Section 3.3): the discrepancy maps m_1, m_2, m_3 of the three teacher scales are combined by non-negative weights, and the resulting map is reduced to a scalar. The published pipeline fixes both choices implicitly: uniform weights, and the arithmetic mean over all pixels.

Neither is forced by the method. We therefore treat the pair $(\mathbf{w}, \rho)$ as a *scoring policy*, with $\mathbf{w}$ on the probability simplex over three scales and $\rho \in \{\text{mean}, \text{top-}k, \max\}$, where top-$k$ averages the k highest-scoring pixels with $k = 1\%$ of the map. We evaluate 82 policies: the 66-point simplex grid at step 0.1 crossed with top-k pooling, plus 9 single-scale policies (`s1`, `s2`, `s3` $\times$ mean, top-k, max), 3 policies that sum the three maps without weights, and 4 that normalise each map before combining

$(66 + 9 + 3 + 4 = 82)$. The exact enumeration, with every $(\mathbf{w}, \rho)$ pair keyed by name, is released in `fusion_policy/fusion_summary.json`.

A second and larger policy set is used in Section 5.5, where the search over policies has to be richer because the criterion rather than the pooling is under test: the finer simplex grid at step 0.2 is paired with all three reductions, and the coarser step-0.1 grid is paired with top-k pooling, together with the three canonical uniform variants, giving $66 + 21 + 21 + 3 = 111$ policies. The two sets are not interchangeable, and the numbers below are quoted per set; where they are compared directly, Table 6 says so.

An important property of this axis is that a fixed grid of it can be evaluated without retraining: changing $(\mathbf{w}, \rho)$ requires no new parameters and no extra inference, since the discrepancy maps are already computed and pooling is a reduction over them, so every policy in the grid is evaluated on the *same* trained model. The simplex is continuous, so no finite grid can be exhaustive; what we evaluate is a fixed grid of the size stated above, and the sensitivity sweep of Section 5.2 is what establishes that the grid is fine enough for the conclusion. This is a practical convenience for deployment — the policy can be re-tuned on an existing detector without repeating the training run — and it is what makes the grid evaluation below affordable.

It is worth being explicit that this does not diminish the role of training, and we do not present the two axes as competing. Establishing that the architecture axis is inert (Section 4) required retraining hundreds of detectors, and we regard that expense as the necessary price of a defensible claim. The axes are complementary: training determines what a model can represent, and the scoring policy determines how much of that is read out.

5.2 Exhaustive Evaluation on a Fixed Model

We train one detector per MVTec-AD category with the protocol used throughout this paper (EfficientNet-B0 student, 500 epochs, batch size 64, seed 42, ResNet-18 teacher), then evaluate all 82 policies on the frozen model. Table 5 reports the result.

Table 5 Scoring policies evaluated on one frozen detector per category (15 categories, 82 policies). "Worst category" is the lowest per-category AUROC, i.e. the failure mode that matters in deployment. The default is the policy the published pipeline uses. The per-category oracle *reads the test labels* to pick the best policy for each category and is an upper bound, not an achievable result; every other row is label-free.

Policy	Mean AUROC (%)	Worst category (%)	Extra retraining
`mean`, uniform weights (default)	82.80	**42.22**	
`max`, uniform weights	86.84	—	none
`top-k`, uniform weights	90.21	78.37	none
best single policy over the grid (`w0.3_0.6_0.1`, top-k; picked on test, so also an oracle)	**90.60**	—	none
per-category oracle over 82 policies (uses labels)	**93.72**	84.67	none

Switching the pooling from the arithmetic mean to top-k raises mean AUROC from 82.80% to 90.21%, a gain of 7.42 points, with no change to the model. (Throughout this section the deltas are computed on the unrounded values and rounded only for display, so a delta may differ by 0.01 from the subtraction of the two printed figures — here $90.2075 - 82.7896 = 7.4179$.) Scale weighting adds much less: the best weighted policy reaches 90.60%, only 0.39 points above the uniform top-k. Every one of the eight highest-scoring policies uses top-k pooling, and all of the improvement therefore comes from the reduction step rather than from the scale weights. Choosing the policy per category raises the mean further, to 93.72%— 10.92 points above the default, and again without retraining anything.

The pooling effect is not an artefact of one training run. Repeating the comparison with two further seeds (43 and 44) gives +8.07 and +7.78 points against +7.42 for the seed reported here; the per-seed estimates span 0.65 points. Aggregating the two later seeds alone gives policy means of 82.36% for the default and 90.28% for top-k (+7.92 points, seeds 43–44). We quote the per-seed estimates and that within-script aggregate rather than one summary number, because seed 42's models enter the reported table through a different evaluation script than the later two do (Table 6): averaging across the two scripts would average two implementations of the same statistic, which we decline to do, so we do not quote a single three-seed mean for this quantity. Throughout, the seed reported in full is seed 42, on which the default and top-k read 82.80% and 90.21% (+7.42); the headline we carry into the abstract and conclusion is the conservative +7.5 points, the mean on the label-free 70/30 split of Section 5.5. Every quantity we can measure for this effect therefore lies between +7.4 and +8.1 points, and we quote the smallest of them as the headline.

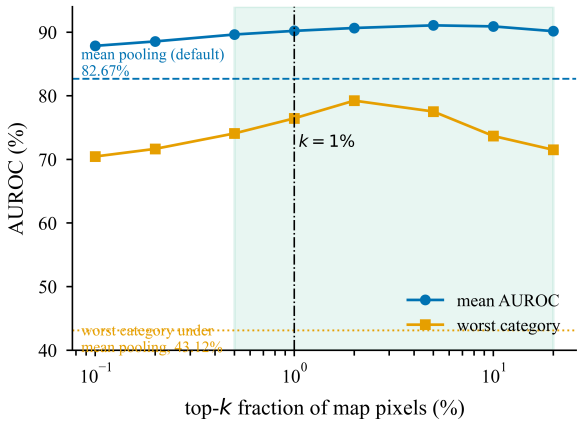


Fig. 5 Sensitivity of the pooling gain to the top-k fraction. The effect is a broad plateau, not a spike: every k between 0.5% and 20% recovers 7 to 8.4 points of mean AUROC relative to mean pooling, and the published choice of $k = 1\%$ lies inside the plateau rather than on its edge. The worst category peaks earlier, at $k = 2\%$.

The effect is not a hyper-parameter artefact. The comparison fixes $k = 1\%$ of the map, so it is fair to ask whether the gain is an artefact of that choice. Sweeping k over two orders of magnitude (Table 6 and Figure 5) traces a broad plateau rather than a spike: mean AUROC rises from 86.74% at $k = 1$ pixel to a maximum of 91.08% at $k = 5\%$, and stays above 89.6% for every k between 0.5% and 20%. The default of 1% sits inside the plateau, not on its edge, and any value in that range recovers 7 to 8.4 points. The worst-category statistic peaks earlier, at $k = 2\%$ (79.24%), so k between 1% and 2% is the best compromise between average accuracy and worst-case behaviour.

What the pipeline's own noise floor is. The default policy's mean AUROC has now been measured three times on this pipeline: 82.80% here, 82.40% in an earlier independent training run of the same configuration, protocol and seed, and 82.67% by the second evaluation script behind Table 6. The three agree to within 0.4 points, and the two that share the same trained models — the first and the third — to within 0.13 points on the aggregate. We report this rather than average it away, because it sets the scale against which the effects in this section should be read: the pooling gain is 7.42 points, an order of magnitude larger than the 0.13–0.4 point aggregate spread of repeated measurement of the baseline. It also

Table 6 Sensitivity of the pooling gain to the top-k fraction, uniform weights, 15 categories, computed by a second evaluation script on the same seed-42 trained models. Every row in this table shares those maps, so the comparison is internally exact. The two scripts *disagree per category* even though their 15-category means agree: for the identical policy (`sum_topk` in the first script, `uniform|topk_0.01` in the second) the two read 90.21% both, but their per-category values differ by a mean absolute 0.74 points and up to 2.49 points (`tile`: 85.79 vs 88.28), and the worst-category statistic — the stated rationale for the k range recommended below — differs by 1.91 points (`pill`: 78.37 vs 76.46). The default policy reads 82.67% here against 82.80% in Table 5, a 0.13-point aggregate difference that is the same level at which the two implementations agree on this pipeline. We keep both scripts in the release and quote the range rather than select one, and we flag that the per-category and worst-category numbers in this section should be read as implementation-dependent to about two points. Two more such pairs, for completeness: max pooling reads 86.74% here against 86.84% in Table 5, and the best weighted policy reads 90.65% here against 90.60% there.

Reduction	Mean AUROC (%)	Worst category (%)	Δ vs. mean pooling
mean over all pixels (default)	82.67	43.12	—
1 pixel (= max)	86.74	68.52	+4.07
$k = 0.1\%$	87.86	70.46	+5.19
$k = 0.5\%$	89.64	74.09	+6.97
$k = 1\%$	90.21	76.46	+7.54
$k = 2\%$	**90.66**	**79.24**	+7.99
$k = 5\%$	**91.08**	77.53	+8.41
$k = 10\%$	90.92	73.68	+8.25
$k = 20\%$	90.17	71.51	+7.50

illustrates the paper's central point at a smaller scale: two implementations of the same statistic on the same models agree to 0.13 points in their 15-category mean — a quarter of the 0.5-point margin the field contests — while disagreeing per category by up to 2.49 points (Table 6). Repeating a measurement with a second script therefore moves the aggregate by a negligible amount and the individual numbers by more than the margin being contested, which is itself an argument for reporting the resolution of a design alongside its results. The per-category oracle, by contrast, is an upper bound that requires anomaly labels and is reported to bound what a selection procedure could achieve, not as an attainable number.

Catastrophic failures are a scoring artefact. The mean column of Table 5 understates the practical importance of the pooling choice, because the default policy does not merely lose accuracy on average — it fails catastrophically on specific categories. Under top-k pooling the worst category rises from 42.22% to 78.37%, and under the per-category oracle to 84.67%. The clearest case is `screw`, where the default policy scores 42.22%, below random guessing, while the

same trained model scores 84.01% under top-k pooling and 87.76% under the best per-category policy — a gain of 41.8 and 45.5 points respectively. This deserves emphasis because the `screw` failure reads like a property of the detection paradigm, whereas it is produced by the reduction step. Averaging the discrepancy map over all pixels destroys the signal whenever a defect occupies a small fraction of the image, because the few genuinely anomalous pixels are diluted by the vast majority of normal ones. `screw` consists largely of thin, localised thread defects, exactly the regime in which a mean over the map is guaranteed to lose. The same mechanism drives the other large top-k gains: `metal_nut` +19.65, `grid` +18.71, `toothbrush` +13.61, `carpet` +13.44 and `capsule` +11.61 points. Under per-category selection the largest additional gains are `grid` +27.90, `carpet` +21.55 and `metal_nut` +21.41 points over the default. The pooling choice is therefore not a refinement of a working method: it is the difference between a detector that fails below chance on a standard benchmark category and one that works on it, and it requires no retraining at all.

Scope of the pooling result. The comparison above is measured on one backbone family: the detectors carrying the 82 policies are EfficientNet-B0 students (Section 5.2). Section 5.3 repeats the sweep on a second backbone (ResNet-18) and on a second benchmark (VisA), and both replications hold, but we have not repeated it across the remaining thirteen backbones of the search space, so the effect is established on two detector families rather than on all fourteen. What that experiment shows is that the gain is neither universal nor arbitrary: it follows from how badly the mean pooling dilutes the defect, and the crossover predicts, category by category, where it should fail.

It is also not the case that top-k pooling dominates the mean everywhere. It helps strongly on the six categories just listed, but it is slightly worse on `transistor` (−2.00), `zipper` (−3.02), `tile` (−4.62), `pill` (−4.80) and `cable` (−5.06) points, on the same trained models. The two poolings therefore respond to different defect statistics, which is why per-category selection adds a further 3.5 points on top of the uniform top-k.

5.3 Why the Gain Appears, and When It Does Not

The effect replicates on a second benchmark. Every measurement in Section 5.2 is on MVTec-AD. We repeat the protocol unchanged on VisA [41]: EfficientNet-B0 students, the same ImageNet ResNet-18 teacher, 500 epochs, batch size 64, seed 42, one detector per category over the twelve VisA categories, and the same 82 policies. The published default reaches 82.35% mean AUROC and the fixed top-k convention 90.57%, a gain of +8.22 points against +7.42 on MVTec-AD, with nine of the twelve categories improving. The effect is therefore not a property of one benchmark, and it is the same effect rather than a coincidentally similar mean: the ordering of the 82 policies agrees between the two datasets at Spearman $\rho = 0.77$ ($p = 3 \times 10^{-17}$), and the eight strongest policies are top-k based on both.

A second student family. Replacing the EfficientNet-B0 student with a ResNet-18, under the same teacher, protocol and policy grid, raises mean AUROC across the fifteen MVTec-AD categories from 83.14% to 91.14% (+8.00 points, ten categories improving). The more informative result is not the mean but the pattern: the per-category gains agree between the two backbones at Spearman $\rho = 0.98$ ($p = 3 \times 10^{-10}$), and the same ten categories improve while the same five do not. Which categories benefit is therefore a property of the category, not of the network — what a dilution argument predicts, and what a backbone-specific artefact would not.

It is not uniform, and the exceptions are not arbitrary. The same change loses on three VisA categories (`candle` −6.6, `capsules` −4.8, `fryum` −0.3) and on five MVTec categories, and gains more than sixteen points on six others. Twenty-nine of the forty-two category runs improve and thirteen do not, so a rule that merely asserts that top-k helps would be wrong in nearly a third of the cases. What separates the two groups is not the defect size, as one might expect, but how much room the default pooling leaves.

A dilution argument. Write the discrepancy map as $\{m_i\}_{i=1}^{P}$, let a be the fraction of the image the defect covers, δ the amount by which it raises the map, and σ the pixel-level spread of the map

on normal images; let $s = \delta/\sigma$ be the per-pixel signal-to-noise ratio. The mean averages P_{eff} effectively independent pixels, so the two populations are separated by $a\delta$ against a standard error of $\sigma/\sqrt{P_{\text{eff}}}$:

$$d'_{\text{mean}} \;=\; a\, s\, \sqrt{P_{\text{eff}}}. \tag{5}$$

The mean therefore dilutes *linearly* in the defect area: halving the defect halves the separation, however strong the defect is. This is exactly the failure repaired on `screw` in Section 5.2, where $a \approx 0.3\%$.

Top-k reads the tail instead. On a normal image that tail is noise, whose expected height is $c(k)\sigma$ with $c(k) = \varphi(z_{1-k})/k$ — 2.67 at $k = 1\%$ and 2.06 at $k = 5\%$ — while on an anomalous image the top $K = kP$ pixels are defect pixels as soon as $a > k$, so the separation saturates at the defect amplitude minus that floor:

$$d'_k \;=\; \big(s - c(k)\big)\sqrt{kP/\rho}, \tag{6}$$

with $\rho = O(1)$ the variance inflation of a K-term mean relative to σ^2/K. Equations (5) and (6) differ in kind: one grows with the defect area and the other does not, and the second pays a floor that the first does not. Dividing them, top-k is the better estimator exactly when

$$a \;<\; a^\star(k, s) \;=\; \Big(1 - \frac{c(k)}{s}\Big)\sqrt{k/\rho}, \tag{7}$$

which at $k = 1\%$ and $\rho \approx 1$ reads $a^\star \approx 0.1\,(1 - 2.67/s)$: about 1.1% of the image at $s = 3$, 5.6% at $s = 6$, and 7.3% at $s = 10$. The crossover sits at a few percent of the image, which is the range that separates sparse defects (`screw` 0.34%, `cashew` 1.7%) from extended ones.

The link to AUROC is concave, which sets the size of the effect. The separations do not reach the reported number linearly. For approximately Gaussian scores $\text{AUROC} = \Phi(d'/\sqrt{2})$, so $\partial\,\text{AUROC}/\partial d' = \varphi(d'/\sqrt{2})/\sqrt{2}$ vanishes as d' grows: a change in d' buys most where the default is weak and almost nothing where it is already close to one, and a category on the wrong side of (7) loses only what the flat part of Φ leaves to lose. Two consequences are testable and both hold.

First, the gain should decrease with the AUROC the published default already attains. Over the forty-two category runs — fifteen MVTec-AD categories with an EfficientNet-B0 student, the same fifteen with a ResNet-18 student, and twelve VisA categories with the EfficientNet-B0 student — it does, at Spearman $\rho = -0.67$ ($p = 1 \times 10^{-6}$; Figure 6a), with mean gains of +28.9, +10.3, −0.7 and +0.9 points in the four baseline bands of that panel. The largest gain (+41.8) is also six times the largest loss (−6.6), which is the asymmetry a concave link predicts and a symmetric additive effect would not.

Second, defect area should matter only through the product $a\,s$, and the baseline already measures that product. Conditional on the baseline the area should therefore add nothing, and it does not: over the forty-two runs the marginal correlation of the gain with the median defect area of the category is $\rho = -0.29$ ($p = 0.06$), and the partial correlation controlling for the baseline is +0.06 ($p = 0.71$; Figure 6b). Defect area is a cause, but only through the dilution term, and that term is already visible in how well the default performs.

What this changes for practice. The pooling axis is not a free lunch to be switched on everywhere. It is a repair for one specific failure — a mean that dilutes a sparse defect — and it costs a small, bounded amount where that failure does not occur. In deployment terms: apply it when the defect occupies a small fraction of the frame, and expect the largest gains on exactly the categories where the default currently fails. Both statements follow from (7) rather than from these two benchmarks, and both are falsifiable on a new category by measuring its baseline AUROC and its defect area, without training anything.

5.4 Can the Policy Be Chosen Without Labels?

The per-category oracle is not available in deployment, since it uses anomaly labels. The practical question is whether $(\mathbf{w}, \rho)$ can be selected from normal data alone, which would make the gain available without ever using an anomaly label.

Normal-distribution criteria fail. A natural first attempt is to choose the policy under which normal training images look most tightly clustered, or whose score distribution has the lightest tail. We tested six such criteria on a held-out portion of the normal training data, for a fixed model and its own held-out split; the default and oracle

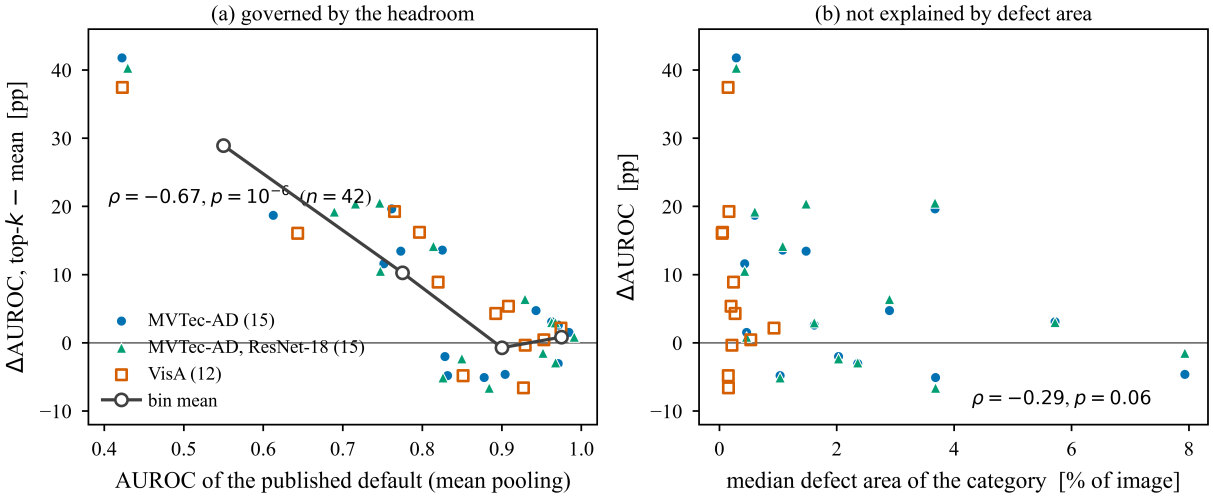


Fig. 6 The pooling gain is governed by the headroom, not by the defect size. (a) Change in mean AUROC when the published default is replaced by the fixed top-k convention, against the AUROC the default already attains, over 15 MVTec-AD categories with an EfficientNet-B0 student, the same 15 with a ResNet-18 student, and 12 VisA categories; the line joins the means of four baseline bands. (b) The same change against the median ground-truth defect area of the category. Defect area alone does not predict the gain, and adds nothing once the baseline is held fixed (partial Spearman +0.06, $p = 0.71$).

figures quoted in this paragraph are that experiment's own (82.39% and 93.39% on its split, on a subset of the 82-policy grid), so they are not directly comparable with the 82.80%/93.72% of Table 5, which uses the full training set and the full grid. All six do worse than the default: the best reaches 82.28% against a default of 82.39% (−0.12 points) and the worst 81.12% (−1.28 points), while the oracle over the same policies is 93.39%; the criteria select the oracle policy in only 1–2 of 15 categories. The reason is structural. Top-k pooling deliberately amplifies the upper tail of the discrepancy map, and on a normal image this tail is noise, so top-k makes normal data look *less* well behaved, not more. Criterion and objective are therefore anti-correlated: the property we would like a clean image to have is not the property that makes a defective image detectable. Tightness of the normal distribution is the wrong proxy, and no amount of tuning fixes a proxy measured on the wrong side of the problem. The failure is one of criterion rather than of estimation, so we replace it with a discriminative one.

5.5 Discriminative Selection from Synthetic Anomalies

Held-out normal images are perturbed with synthetic anomalies—the device used by DRAEM-style synthesis [22]—and we select the policy that best separates the perturbed images from their clean originals. Three families of perturbation are used (smoothed blobs, thin oriented scratches, and rectangular cuts), each with a per-image randomised strength, five seeds per family, giving fifteen perturbed variants of the holdout set. Selection maximises the area under the ROC curve between clean and perturbed holdout images. Only `train/good` images are ever touched, so the procedure never sees a real defect; and because it operates on a trained detector, adopting its output requires no retraining.

Table 7 reports the outcome over all 15 categories and three training seeds, and Figure 7 the per-category detail.

The rule that carries the gain is the pooling, not the selection. On the same models and the same split, simply fixing top-k pooling — which needs no criterion, no pseudo-anomalies and no selection step — reaches 88.99% on average against 81.50% for the default and 88.59%–88.78% for the four pseudo-anomaly rules. The pseudo-anomaly criterion therefore does not beat the fixed

Table 7 Label-free selection of the scoring policy on 15 MVTec-AD categories, from 111 candidate policies, repeated on three independently trained students. Every row is measured on the same trained models within a column; no row above the oracle requires a real anomaly label. Columns are individual seeds, because they do not agree (see the text).

Selection rule	Seed 42	Seed 43	Seed 44	Mean
default policy (`mean`, uniform)	81.62	81.90	80.98	81.50
fixed ρ = top-k, no selection (baseline)	**88.90**	**89.44**	**88.62**	**88.99**
pseudo-anomaly separation, best policy	89.99	89.96	85.81	88.59
pseudo-anomaly separation, top-5 ensemble	89.96	90.03	86.08	88.69
pseudo-anomaly separation, top-10 ensemble	89.70	89.98	86.25	88.64
pseudo-anomaly separation, top-20 ensemble	89.34	89.72	87.28	88.78
per-category oracle (uses labels; upper bound)	93.19	93.36	92.90	93.15
oracle top-5 ensemble (upper bound)	93.81	93.52	93.12	93.48

convention on average; it is a refinement whose own contribution is −0.40 points, and its four variants all land in a 0.2-point band that lies *below* the fixed convention rather than around it.

The procedure at its best policy returns 89.99% on seed 42, 89.96% on seed 43 and 85.81% on seed 44, against the table's default row of 81.62, 81.90 and 80.98% — a mean gain of **+7.09** points over the default, and **61**% of the +11.65 points that per-category oracle selection would have delivered on the same models.

But most of that gain is not the criterion's. Reporting the gain against the default policy alone would overstate the method, and we therefore decompose it. The 61% figure answers "how much of the oracle gap does the label-free procedure close?", but it does not answer "how much of that does the *criterion* contribute?", and the two questions have very different answers. The decomposition here is computed on this subsection's 70/30 split, so its fixed-top-k gain (+7.49) is not the full-test-set figure (+7.42 on seed 42, +7.78 to +8.07 on the two later seeds) quoted in Section 5.2; the two differ because the split does, and we report both rather than translate one into the other. On the same 70/30 split and the same trained students, replacing the default with a *fixed* top-k policy — that is, adopting the pooling recommendation of Section 5.2 and choosing no weights at all, with no selection and no pseudo-anomaly data — already reaches **88.90**% on seed 42, against 81.62% for the default and 89.99% for the full procedure. Across the three seeds the fixed top-k convention gains +7.28, +7.54 and +7.64 points (mean **+7.49**), while the pseudo-anomaly criterion adds a further +1.10, +0.52 and **−2.82** points over that fixed convention (mean **−0.40**). In other words, of the +7.09 points attributed above to the label-free procedure, roughly +7.49 comes from the plain decision to pool by top-k rather than by the mean, and the criterion's own contribution averages slightly *below zero*: on two of three seeds it is a small positive refinement, and on the third it is worse than simply fixing k. We report this rather than let the larger number stand on its own, because the honest reading of the evidence is that the useful, label-free recommendation of this paper is *the pooling*, and that the selection criterion is an unproven refinement of it. The decomposition is released as `pseudo_select/baseline_decomposition.py` and its JSON output.

This reading is reinforced by the control that filtering degenerate trials does not rescue the criterion (Section 4.5), and by the fact that the criterion's three-seed spread (+1.10 to −2.82 over the fixed convention) is itself larger than its mean effect. The comparison is within-model throughout, so the +7.49 is not a split artefact: it is the pooling change measured on the same students.

The three seeds are worth reporting individually because they do not agree: the gain over the default is +8.37, +8.07 and +4.83 points on seeds 42, 43 and 44. The oracle gap is nearly constant across seeds (11.46–11.92), so the variability lies in the criterion rather than in what is available to be recovered. This is the lesson of Section 4.4 at a smaller scale: a three-seed estimate of an effect this size is a range, not a number, and a single-seed report of the label-free gain would have claimed 8.4 points where the honest three-seed figure is 7.1.

Two features of the procedure are stable even though its headline gain is not. The pseudo criterion ranks policies by their true AUROC at a mean Spearman $\rho = 0.53$ in *every* seed (0.529–0.536): the ordering it learns is reproducible, while the policy at the top of that ordering is not. And the criterion selects the per-category oracle policy in only one of the forty-five category-seed cells, yet still attains 88.59% on average — it does not need to identify the optimum, only to move near-optimal policies to the front. Ensembling the top five policies matches the single choice on average (88.69 against 88.59) and is better in the weakest seed (86.08 against 85.81), so it is the safer default for a criterion with this much seed variance. Averaging over more seeds and perturbation families is

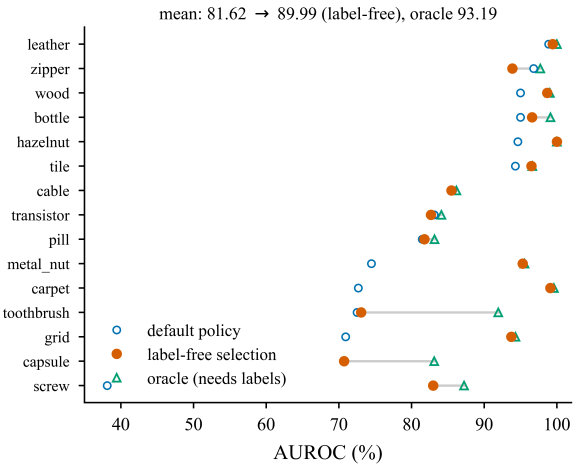


Fig. 7 Label-free selection per category (seed 42). Hollow markers are the published default policy, filled markers the policy chosen from synthetic pseudo-anomalies without any anomaly label, and triangles the per-category oracle. The label-free procedure recovers most of the available gap, and most of it on the categories where the default policy fails badly (`screw`, `grid`).

the obvious way to tighten it, and we leave that to future work.

The comparison in Table 7 is measured on a student trained on 70%

of the normal training images, with the remaining 30% held out for selection (the seed-42 default row is therefore 81.62% rather than the 82.80% of Table 5, which uses the full training set). The gain is a within-model comparison and is not inflated by the split; absolute numbers, however, should be read with the split in mind.

Why the discriminative criterion works where the distributional one fails. The two criteria differ in what they ask of a clean image. The distributional criterion asks that clean images look homogeneous, which top-k pooling actively defeats. The discriminative criterion asks only that the policy separate perturbed from clean — the same question the detector is ultimately asked at test time, posed on synthetic defects. It is a weaker and better-aimed question, and it is answerable from normal data alone. Its own variance is the open problem, not its direction.

6 Discussion: Where This Pipeline Stands

This paper reports a diagnostic study, not a new state-of-the-art detector, and it is therefore necessary to state plainly where the pipeline of Section 3 stands. Two comparisons are relevant. As *reported by the original authors*, image-level AUROC on MVTec-AD runs from 78.0% (AutoEncoder) and 85.0% (VAE) [13] through 95.6% (STPM) [20], 96.0% (SPADE) [18], 97.9% (PaDiM) [17], 98.5% (reverse distillation) [1], 99.1% (PatchCore [4], EfficientAD [23]) and 99.3–99.6% (CFA [24], FastFlow [19], SimpleNet [5], Dinomaly [6]). FastFlow reports 99.4%. These numbers were not re-run here and their protocols are not identical to ours, so they are for orientation only. Two of them, SPADE and FastFlow, are preprints that were never published at a venue; they are included because they are standard baselines in this literature, and their preprint status is declared here rather than left implicit. The second comparison, Table 8, is our own, in which all 14 backbones of the search space were trained under one protocol on all 15 categories.

The pipeline described in this paper reaches 82.47% mean AUROC on MVTec-AD with the architecture the search selected, `efficientnet_b0` (5.41M parameters), against reported values of 95.6%–99.6% for the established detectors. We therefore do not claim a competitive operating point: the gap to the strongest published single-class detectors is 17.1 points, and we report it rather than obscure it. We can also name at least one candidate cause. The deployed detector uses an ImageNet-1k ResNet-18 as its frozen teacher (Section 3.1), whereas reverse distillation as published pre-trains the teacher on the normal images of the target category; the same code path also fixes the student bottleneck at a width the search cannot change. We do not claim that these account for the whole gap, and we have not run the teacher-adapted variant, so the attribution is a hypothesis rather than a result. A second caveat is that the harness itself was not validated against a published reverse-distillation configuration: we did not reproduce a reference RD number inside our code path, so the 17.1-point gap could in principle contain a harness-level difference (for instance the batch-norm behaviour of the largest backbones,

Table 8 Controlled comparison: every backbone of the search space trained on every MVTec-AD category under an identical protocol (500 epochs, batch size 64, seed 42, ResNet-18 teacher, reverse distillation; 210 detectors). Deployed size is the student encoder plus decoder. The selected architecture is marked with †. Mean AUROC and Rank are single-seed (seed 42) and should be read with the resolution of Section 4.4 in mind: *every* adjacent rank differs by less than the 2.98-point minimum detectable difference (the largest adjacent gap is 0.75 points), and only non-adjacent extremes — for instance rank 1 against rank 13, a gap of 3.23 points — exceed it. The ordering is therefore resolved only at its extremes and unresolved across most of its length. Six of the fourteen backbones were replicated at two further seeds; among those six the ordering is *identical* in all three seeds (`resnet18` > `efficientnet_b0` > `mobilenetv3_small_100` > ViT-Tiny > ConvNeXtV2-Tiny > Swin-Tiny, maximum rank movement 0), which is the evidence that the axis is real but too small to be ranked by a single-seed comparison of the other eight.

Backbone	Student (M)	Mean AUROC (%)	Rank
ResNet-18	13.04	**83.34**	1
ResNet-34	23.15	82.99	2
ResNet-50	25.77	82.75	3
MobileOne-S0	6.27	82.58	4
EfficientNet-B0†	5.41	82.47	5
DeiT-Tiny	7.31	82.03	6
MobileNetV3-Small	2.81	81.85	7
ViT-Tiny	7.31	81.51	8
ConvNeXt-Small	51.39	81.43	9
ConvNeXtV2-Tiny	29.80	81.35	10
ConvNeXt-Tiny	29.75	80.91	11
EfficientNet-B1	7.92	80.43	12
Swin-Tiny	29.26	80.11	13
ConvNeXtV2-Base	89.69	79.36	14

which are trained with gradient accumulation and a smaller micro-batch; Section 3.4) on top of the teacher choice. We report this rather than present the comparison as exact, and we note that it does not affect the paper's diagnostic claims, which are all within-harness. Nor is the gap explained by the parameter budget: Section 4.3 varies the deployed size over a 9.2× range across five capacity bands and finds no statistically resolvable effect, and across the full space of Table 8 the deployed size spans 32× while the backbone accounts for 0.54% of the variance. One caveat applies to the reported values: they come from implementations, augmentation policies and training schedules that differ from ours, so the comparison is indicative rather than exact. Table 8 is the within-study comparison on which the analysis in Section 4 rests; it was produced by the same code path, on the same data split, with one variable, the backbone, changed at a time.

Two things are worth taking from Table 8 in their own right. It is, to our knowledge, the first comparison of a full anomaly-detection search space under a single protocol, and as such it is a reusable resource: the ordering it establishes is what makes the negative result in Section 4 measurable rather than anecdotal. And it shows that the search's output, at rank 5 of 14, is a reasonable but unremarkable choice from a space whose top eleven candidates span 2.43 points. The canonical headline number of this paper is the 82.47% mean AUROC of `efficientnet_b0` on the fifteen categories, read from the seed-42 truth matrix (`proxy_diag/phaseB_truth.json`). An earlier, independently trained run of the same configuration obtained 82.40% on the same fifteen categories, a difference of 0.07 points. We report that agreement as a stability check and not as confirmation: 0.07 points is far below this paper's own 2.98-point minimum detectable difference, so the two numbers are simply *consistent with* a stable protocol, and the sentence cannot be made stronger without contradicting Section 4.4. The search itself does *not* reproduce as closely; Section 4.2 documents why.

7 Conclusions

This paper belongs to the class of studies that measure how a field evaluates itself. Such studies do not add a detector; they establish what the field's existing measurements are worth, and they are useful to the extent that the answer changes how the next comparison is read. Our answer is that the standard evaluation design for unsupervised anomaly detection cannot resolve the differences the field reports, and that in the pipeline we instrumented the design axis those comparisons are usually about contributes almost nothing.

The binding constraint is resolution. A two-way decomposition of the 210-detector matrix we trained leaves a residual standard deviation of 3.30 points, and the run-to-run standard deviation of a single detector at a fixed (backbone, category) is 1.96 points over three seeds. Comparing two backbones across the fifteen categories the benchmark provides is therefore a paired test whose minimum detectable difference is 2.98 points — 75% of the entire 3.98-point spread of the fourteen backbones, and six times the 0.5-point margin that

separates PatchCore from the strongest results reported since. Resolving that margin on a single category would take at least 241 independent training runs per backbone, or 533 categories. This is a statement about the benchmark rather than about our pipeline, and it bounds every architecture comparison made on it.

Within that constraint, the diagnosis of the architecture axis is unambiguous. Two thirds of the search space is spent on a feature dimension that cannot reach the deployed model, because the encoder that consumes it hard-codes a width while the decoder honours the requested one. The search signal does not distinguish architectures: it separates the twelve degenerate-free backbones by 5.7% of its median, which is the same order as its own run-to-run noise, and it returns exactly 0.0 when a student diverges — for two backbones in most trials, and occasionally for any of them. And the quantity it is used to predict is dominated by something else entirely: architecture accounts for 0.54% of the variance in detection accuracy and the category for 95.32% (Section 4.3).

The interaction between the first two facts produces the result we did not anticipate. Because the objective can silently return the optimum of its own range, and the search minimises a single sample per candidate, the rule does not select the best architecture but the one with the heaviest left tail — which is the one that failed. Twelve independent runs selected a configuration prone to this collapse every single time, and the architectures they returned perform *worse* than uniform random selection, at the 1.2th percentile of the random distribution (Section 4.5). The four older runs whose results this paper reports chose a backbone that does not collapse, which on this account was luck; those four are a different, earlier run population, and Section 4.5 states the two populations separately so that the record is not read as self-contradictory.

The constructive finding is where the accuracy actually is. The reduction that turns a multi-scale discrepancy map into an image-level score is worth +7.4 points of mean AUROC on MVTec-AD, +8.0 with a second student family and +8.2 on a second benchmark, none of which required retraining anything; the first is nearly twice the entire 3.98-point spread of the fourteen backbones the architecture axis ranks. The gain is not uniform, and Section 5.3 derives why: the mean dilutes linearly in the defect area while top-k saturates at the tail floor of the map, so the two cross over at a defect size of a few percent and a concave link to AUROC decides how much either side of that crossover is worth. That gain is attributable to a fixed change of pooling, with no selection step at all. The default choice, an unweighted mean over all pixels, is not merely suboptimal: it fails below chance on `screw`, and top-k pooling repairs that failure to 84%. The gain is robust across two orders of magnitude in k (Section 5).

Selecting the pooling per category with a label-free criterion we propose does *not*, on the evidence we collected, add materially to the fixed choice. On the same held-out split and the same trained students, fixing top-k pooling already reaches 88.99% on average against 81.50% for the default, while the pseudo-anomaly criterion adds +1.10, +0.52 and −2.82 points on the three seeds (mean −0.40) on top of that fixed convention. We therefore present the pooling change as the finding, and the selection criterion as an unproven refinement of it whose own three-seed spread (3.92 points) exceeds the magnitude of its own mean (0.40 points) (Section 5.5).

Three limitations bound these claims. The diagnosis of the architecture axis is established on MVTec-AD, with one teacher per track (the contrastive proxy teacher and the frozen ImageNet deployed teacher described in Section 3.1) and one training protocol, although the scoring-policy result of Section 5.3 is repeated on a second detector family and on a second benchmark with the same protocol; whether the same collapse appears in other pipelines is a question our evidence cannot settle, though the mechanism — a thresholded objective that can return its own optimum, combined with a minimising selection rule — is not specific to this codebase. Our recommendation for the architecture axis is deliberately narrow: the backbone effect is statistically detectable ($p = 0.039$) and merely two orders of magnitude smaller than the category effect, so we do not conclude that architecture does not matter, nor that NAS for anomaly detection is worthless in principle. And the resolvability figure is conditional on this benchmark's fifteen categories and this task's residual variance; it should be recomputed rather than assumed for another benchmark

or detector family — which is exactly the calculation a benchmark paper ought to publish, and which here decides the question.

Two things follow for practice. A search whose objective can silently return the optimum of its own range must filter degenerate evaluations before minimising, and should report the distribution of its per-configuration scores rather than a single minimum. And a reverse-distillation detector of the family tested here, currently scoring images by the mean over its discrepancy map, can be re-scored on the models it already has with a k of two percent, which on the fifteen categories we measured gained 7–8 points of mean AUROC and removed the below-chance failures. We state this as what we measured rather than as a guarantee for every pipeline: the pooling gain is established on one detector family and should be re-measured before being relied on elsewhere. Neither recommendation requires convincing anyone that a new architecture is better, which is convenient, because on this benchmark that cannot be shown.

Acknowledgements. This work was supported by the Guangxi Natural Science Foundation (Grant No. 2025GXNSFHA069076) and the Guangxi University of Science and Technology Doctoral Fund (Grant Nos. 26Z15, 26Z25).

Statements and Declarations

- **Funding**: This research was supported by the Guangxi Natural Science Foundation under Grant No. 2025GXNSFHA069076 (awarded to P.D.) and by the Guangxi University of Science and Technology Doctoral Fund under Grant Nos. 26Z15 (awarded to P.D.) and 26Z25 (awarded to H.L.). The funders had no role in the design of the study, in the collection, analysis or interpretation of data, or in the decision to publish.
- **Competing interests**: The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.
- **Ethics approval and consent to participate; consent for publication; materials availability**: Not applicable. This study did not involve human participants or animals, and no new materials were generated.
- **Data availability**: This work uses only the publicly available MVTec-AD [7] and VisA [41] benchmarks; no new image data were generated. Every derived artefact is released together with the code (see below): the architecture-search logs, the per-configuration proxy evaluations, the 210-detector AUROC matrix, the per-run search records, the auxiliary benchmark harness, the synthetic-mechanism study, and the scripts that regenerate every table and figure. Each number reported in this paper can therefore be traced to the run that produced it.
- **Code availability**: The complete source code and the raw experimental logs are publicly released at https://github.com/fanta12138/kg-nas-diagnosis, under a MIT licence. They comprise the pipeline under study, the diagnostic experiments (proxy repeatability, the truth matrix, the repeated searches), the synthetic reproduction of the degeneracy mechanism, the resolvability analysis, the benchmark harness, and the analysis scripts that produce every table and figure in this paper, together with a SHA-256 manifest of every released file.
- **ORCID**: P.D. https://orcid.org/0000-0002-2892-4251. The remaining authors will supply their identifiers to the editorial office at submission.
- **Author contributions**: All three authors are colleagues in the same school and jointly planned the study. Using the CRediT taxonomy: **Conceptualization**, P.D.; **methodology**, P.D., H.L. and Y.Z.; **validation**, P.D.; **formal analysis**, P.D.; **writing—original draft preparation**, P.D.; **writing—review and editing**, H.L. and Y.Z.; **supervision**, P.D.; **project administration**, P.D.; **funding acquisition**, H.L. and P.D. All authors have read and agreed to the published version of the manuscript. P.D. is the corresponding author.
- **Use of large language models**: An AI assistant (a large language model) was used substantially in this work. It wrote the code for the diagnostic experiments, the synthetic reproduction and the benchmark harness, executed those experiments, and produced the numerical results reported here; it also produced drafts of the manuscript text and the plotting code. No figure or table was generated by a generative model: each figure is a plot of recorded measurements produced by the released script, and

each table transcribes the same recorded result files. The authors specified the research questions, directed the work, verified the reported results against the recorded experimental outputs, and take full responsibility for the content, the claims and their accuracy.

References

[1] Deng, H., Li, X.: Anomaly detection via reverse distillation from one-class embedding. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 9737–9746 (2022). https://doi.org/10.1109/CVPR52688.2022.00951

[2] Qian, C., Lao, X., Li, C.: Knowledge-guided neural architecture search for anomaly localization. In: 2024 IEEE 17th International Conference on Signal Processing (ICSP). IEEE, Piscataway, NJ (2024). https://doi.org/10.1109/ICSP62129.2024.10846006

[3] Xie, L., Yuille, A.: Genetic cnn. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 1388–1397 (2017). https://doi.org/10.1109/ICCV.2017.154

[4] Roth, K., Pemula, L., Zepeda, J., Schölkopf, B., Brox, T., Gehler, P.: Towards total recall in industrial anomaly detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 14318–14328 (2022). https://doi.org/10.1109/CVPR52688.2022.01392

[5] Liu, Z., Zhou, Y., Xu, Y., Wang, Z.: Simplenet: A simple network for image anomaly detection and localization. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 20402–20411 (2023). https://doi.org/10.1109/CVPR52729.2023.01954

[6] Guo, J., Lu, S., Zhang, W., Chen, F., Liao, H., Li, H.: Dinomaly: The less is more philosophy in multi-class unsupervised anomaly detection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 20405–20415 (2025). https://doi.org/10.1109/CVPR52734.2025.01900

[7] Bergmann, P., Batzner, K., Fauser, M., Sattlegger, D., Steger, C.: The mvtec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. International Journal of Computer Vision **129**(4), 1038–1059 (2021) https://doi.org/10.1007/s11263-020-01400-4

[8] Yu, K., Sciuto, C., Jaggi, M., Musat, C., Salzmann, M.: Evaluating the search phase of neural architecture search. In: International Conference on Learning Representations (ICLR) (2020). arXiv:1902.08142

[9] Xie, G., Wang, J., Liu, J., Lyu, J., Liu, Y., Wang, C., Zheng, F., Jin, Y.: IM-IAD: Industrial image anomaly detection benchmark in manufacturing. IEEE Transactions on Cybernetics **54**(5), 2720–2733 (2024) https://doi.org/10.1109/TCYB.2024.3357213

[10] Bouthillier, X., Delaunay, P., Bronzi, M., Trofimov, A., Nichyporuk, B., Szeto, J., Sepahvand, N.M., Raff, E., Madan, K., Voleti, V., Kahou, S.E., Michalski, V., Arbel, T., Pal, C., Varoquaux, G., Vincent, P.: Accounting for variance in machine learning benchmarks. In: Proceedings of Machine Learning and Systems (MLSys). Proceedings of Machine Learning Research, vol. 139, pp. 747–769 (2021). https://doi.org/10.48550/arXiv.2103.03098

[11] Cao, Y., Ma, Y., Zhu, Y., Ting, K.M.: Revisiting streaming anomaly detection: benchmark and evaluation. Artificial Intelligence Review **58**(1), 8 (2024) https://doi.org/10.1007/s10462-024-10995-w

[12] Yu, R., Wang, M., Yun, J., Du, J., Wang, L., Suo, Y.: Unified reproduction and event-level evaluation of industrial multivariate time-series anomaly detection methods. In: 2026 7th International Conference on Computing, Networks and Internet of Things (CNIOT), pp. 1–5. IEEE, Piscataway, NJ (2026). https://doi.org/10.1109/CNIOT69547.2026.11582255

[13] Bergmann, P., Löwe, S., Fauser, M., Sattlegger, D., Steger, C.: Improving unsupervised defect segmentation by applying structural similarity to autoencoders. In: Proceedings of the 14th International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP), Volume 5: VISAPP, pp. 372–380. SciTePress, Setúbal, Portugal (2019). https://doi.org/10.5220/0007364503720380 . Venue and DOI verified against the MVTec publication list and DBLP

[14] Akcay, S., Atapour-Abarghouei, A., Breckon, T.P.: Ganomaly: Semi-supervised anomaly detection via adversarial training. In: Asian Conference on Computer Vision (ACCV), pp. 622–637 (2019). https://doi.org/10.1007/978-3-030-20893-6_39 . Springer

[15] Schlegl, T., Seeböck, P., Waldstein, S.M., Langs, G., Schmidt-Erfurth, U.: f-anogan: Fast unsupervised anomaly detection with generative adversarial networks. Medical Image Analysis **54**, 30–44 (2019) https://doi.org/10.1016/j.media.2019.01.010

[16] Gong, D., Liu, L., Le, V., Saha, B., Mansour, M.R., Venkatesh, S., Hengel, A.v.d.: Memorizing normality to detect anomaly: Memory-augmented deep autoencoder for unsupervised anomaly detection. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1705–1714 (2019). https://doi.org/10.1109/ICCV.2019.00179

[17] Defard, T., Setkov, A., Loesch, A., Audigier, R.: Padim: A patch distribution modeling framework for anomaly detection and localization. In: International Conference on Pattern Recognition (ICPR), pp. 475–489 (2021). https://doi.org/10.1007/978-3-030-68799-1_35 . Springer

[18] Cohen, N., Hoshen, Y.: Sub-Image Anomaly Detection with Deep Pyramid Correspondences. Preprint (2020). https://doi.org/10.48550/arXiv.2005.02357

[19] Yu, J., Zheng, Y., Wang, X., Li, W., Wu, Y., Zhao, R., Wu, L.: FastFlow: Unsupervised Anomaly Detection and Localization via 2D Normalizing Flows. Preprint (2021). https://doi.org/10.48550/arXiv.2111.07677

[20] Wang, G., Han, S., Ding, E., Huang, D.: Student-teacher feature pyramid matching for unsupervised anomaly detection. In: Proceedings of the British Machine Vision Conference (BMVC) (2021). https://doi.org/10.5244/C.35.349

[21] Bergmann, P., Fauser, M., Sattlegger, D., Steger, C.: Uninformed students: Student-teacher anomaly detection with discriminative latent embeddings. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4183–4192 (2020). https://doi.org/10.1109/CVPR42600.2020.00424

[22] Zavrtanik, V., Kristan, M., Skočaj, D.: DRÆM – a discriminatively trained reconstruction embedding for surface anomaly detection. In: 2021 IEEE/CVF International Conference on Computer Vision (ICCV), pp. 8310–8319. IEEE, Piscataway, NJ (2021). https://doi.org/10.1109/ICCV48922.2021.00822

[23] Batzner, K., Heckler, L., König, R.: Efficientad: Accurate visual anomaly detection at millisecond-level latencies. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), pp. 128–138 (2024). https://doi.org/10.1109/WACV57701.2024.00020

[24] Lee, S., Lee, S., Song, B.C.: CFA: Coupled-hypersphere-based feature adaptation for target-oriented anomaly localization. IEEE Access **10**, 78446–78454 (2022) https://doi.org/10.1109/ACCESS.2022.3193699

[25] Liu, J., Xie, G., Wang, J., Li, S., Wang, C., Zheng, F., Jin, Y.: Deep industrial image anomaly detection: A survey. Machine Intelligence Research **21**(1), 104–135 (2024) https://doi.org/10.1007/s11633-023-1459-z

[26] Zoph, B., Vasudevan, V., Shlens, J., Le, Q.V.:

Learning transferable architectures for scalable image recognition. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 8697–8710 (2018). https://doi.org/10.1109/CVPR.2018.00907

[27] Liu, C., Zoph, B., Neumann, M., Shlens, J., Hua, W., Li, L.-J., Fei-Fei, L., Yuille, A., Huang, J., Murphy, K.: Progressive neural architecture search. In: Proceedings of the European Conference on Computer Vision (ECCV), pp. 19–35 (2018). https://doi.org/10.1007/978-3-030-01246-5_2 . Springer

[28] Liu, H., Simonyan, K., Yang, Y.: DARTS: Differentiable architecture search. In: International Conference on Learning Representations (ICLR) (2019). arXiv:1806.09055

[29] Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A., Le, Q.V.: Mnasnet: Platform-aware neural architecture search for mobile. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2820–2828 (2019). https://doi.org/10.1109/CVPR.2019.00293

[30] Real, E., Aggarwal, A., Huang, Y., Le, Q.V.: Regularized evolution for image classifier architecture search. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 33, pp. 4780–4789 (2019). https://doi.org/10.1609/aaai.v33i01.33014780

[31] Ren, P., Xiao, Y., Chang, X., Huang, P.-y., Li, Z., Chen, X., Wang, X.: A comprehensive survey of neural architecture search: Challenges and solutions. ACM Computing Surveys **54**(4), 1–34 (2021) https://doi.org/10.1145/3447582

[32] Gou, J., Yu, B., Maybank, S.J., Tao, D.: Knowledge distillation: A survey. International Journal of Computer Vision **129**(6), 1789–1819 (2021) https://doi.org/10.1007/s11263-021-01453-z

[33] Tung, F., Mori, G.: Similarity-preserving knowledge distillation. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 1365–1374 (2019). https://doi.org/10.1109/ICCV.2019.00145

[34] Liu, Y., Chen, K., Liu, C., Qin, Z., Luo, Z., Wang, J.: Structured knowledge distillation for semantic segmentation. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 2604–2613 (2019). https://doi.org/10.1109/CVPR.2019.00271

[35] Zhang, L., Bao, C., Ma, K.: Self-distillation: Towards efficient and compact neural networks. IEEE Transactions on Pattern Analysis and Machine Intelligence **44**(8), 4388–4403 (2022) https://doi.org/10.1109/TPAMI.2021.3067100

[36] Wang, T., Yuan, L., Zhang, X., Feng, J.: Distilling object detectors with fine-grained feature imitation. In: 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 4928–4937. IEEE, Piscataway, NJ (2019). https://doi.org/10.1109/CVPR.2019.00507

[37] Yang, G., Yu, S., Sheng, Y., Yang, H.: Attention and feature transfer based knowledge distillation. Scientific Reports **13**(1), 18369 (2023) https://doi.org/10.1038/s41598-023-43986-y

[38] Kang, M., Mun, J., Han, B.: Towards oracle knowledge distillation with neural architecture search. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, pp. 4404–4411 (2020). https://doi.org/10.1609/aaai.v34i04.5866

[39] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016). https://doi.org/10.1109/CVPR.2016.90

[40] Woo, S., Debnath, S., Hu, R., Chen, X., Liu, Z., Kweon, I.S., Xie, S.: Convnext v2: Co-designing and scaling convnets with masked autoencoders. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR),

pp. 16133–16142 (2023). https://doi.org/10.1109/CVPR52729.2023.01548

[41] Zou, Y., Jeong, J., Pemula, L., Zhang, D., Dabeer, O.: SPot-the-Difference self-supervised pre-training for anomaly detection and segmentation. In: Computer Vision – ECCV 2022. Lecture Notes in Computer Science, vol. 13690, pp. 392–408. Springer, Cham, Switzerland (2022). https://doi.org/10.1007/978-3-031-20056-4_23